Undergraduate Project Report

2025/26

Network Resource Management Based on Language Model Multi-Agent Systems

Name:	Zheyun Zhao
School:	International School
Class:	2022215124
QMUL Student No.:	221170559
BUPT Student No.:	2022213670
Programme:	Intelligent Science and Technology

Date: 13-04-2026

Table of Contents

Abstract

5G network slicing enables the coexistence of heterogeneous services over a shared radio infrastructure, but allocating limited bandwidth among competing slice types — Enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC) and massive Machine-Type Communication (mMTC) — remains a challenging constrained optimisation problem. This project proposes a Large Language Model (LLM) multi-agent negotiation framework for network slice resource management, in which three specialised slice-manager agents and a global coordinator agent collaborate through a structured proposal–evaluation–negotiation protocol. To reconcile the inference latency of LLMs with the real-time demands of radio scheduling, a two-tier architecture is adopted: the upper tier executes low-frequency multi-agent LLM decisions while the lower tier enforces cached allocations at millisecond granularity. The framework is implemented using LangGraph for agent orchestration and evaluated in a custom Gymnasium-based 5G network slicing simulation environment with clearly defined state space, action space and SLA evaluation logic. Two negotiation strategies — Priority Arbitration and Proportional Compromise — are compared against four baselines (fixed-ratio allocation, threshold-based adjustment, Proximal Policy Optimisation, and single-agent LLM) across steady-state and burst traffic scenarios with five random seeds per condition. Experimental results show that the Proportional Compromise variant achieves average SLA satisfaction rates competitive with the best rule-based baseline (0.606 vs. 0.612 under burst) while outperforming the DRL and single-agent LLM baselines. The negotiation mechanism improves SLA satisfaction by approximately 26% and reduces fairness variance by roughly 50% compared to a no-negotiation ablation, confirming the structural value of the multi-agent protocol. Limitations including decision latency (6–9 seconds per step) and token cost are discussed, and directions for future work including edge-deployed models and hybrid LLM-RL approaches are identified.

Keywords

Network Slicing, Resource Management, Large Language Model, Multi-Agent System, Negotiation Protocol, 5G, Deep Reinforcement Learning, Two-Tier Scheduling

摘要

5G 网络切片技术允许异构服务在共享无线基础设施上共存，但在竞争性切片类型——增强移动宽带（eMBB）、超可靠低时延通信（URLLC）和大规模机器类通信（mMTC）——之间分配有限带宽仍然是一个具有挑战性的约束优化问题。本项目提出了一种基于大语言模型（LLM）多智能体协商的网络切片资源管理框架，其中三个专用切片管理智能体和一个全局协调智能体通过结构化的提案—评估—协商协议进行协作。为调和 LLM 推理延迟与无线调度实时性需求之间的矛盾，采用两层调度架构：上层执行低频多智能体 LLM 决策，下层以毫秒粒度执行缓存的分配策略。该框架使用 LangGraph 进行智能体编排，并在基于 Gymnasium 的自定义 5G 网络切片仿真环境中进行评估，该环境具有明确定义的状态空间、动作空间和 SLA 评估逻辑。两种协商策略——优先级仲裁和比例妥协——与四种基线方法（固定比例分配、阈值动态调整、近端策略优化和单智能体 LLM）在稳态和突发流量场景下进行了比较，每种条件使用 5 个随机种子。实验结果表明，比例妥协变体在突发场景下的平均 SLA 满足率（0.606）与最优规则基线（0.612）具有竞争力，同时优于深度强化学习和单智能体 LLM 基线。协商机制相比无协商消融实验，将 SLA 满足率提高了约 26%，将公平性方差降低了约 50%，验证了多智能体协议的结构性价值。本文讨论了决策延迟（每步 6–9 秒）和令牌成本等局限性，并提出了包括边缘部署模型和 LLM-RL 混合方法在内的未来研究方向。

关键词

网络切片, 资源管理, 大语言模型, 多智能体系统, 协商协议, 5G, 深度强化学习, 两层调度

Chapter 1: Introduction

This report presents the design, implementation and evaluation of an LLM multi-agent negotiation framework for 5G network slice resource management. The framework combines a two-tier scheduling architecture with specialised LLM agents that collaborate through a structured proposal–evaluation–negotiation protocol to allocate shared radio bandwidth among competing slice types. This chapter introduces the research context, states the problem addressed, defines the research objectives and scope, summarises the contributions, and outlines the organisation of the remainder of this report.

1.1 Background and Motivation

Fifth-generation (5G) network slicing enables the creation of multiple logical networks over a shared physical infrastructure, with each slice tailored to a distinct class of service (1). The three canonical slice types defined by the 3rd Generation Partnership Project (3GPP) — Enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC) and massive Machine-Type Communication (mMTC) — differ substantially in their throughput, latency, reliability and connection-density requirements. When these slices share a common pool of radio resources, their Quality of Service (QoS) targets become inherently competitive: heavy bandwidth consumption by eMBB users streaming high-definition video may threaten the latency guarantees required by URLLC users, while periodic mass reporting by mMTC devices can reduce the access resources available to other slices (1).

Traditional approaches to network slice resource management include rule-based heuristic allocation and Deep Reinforcement Learning (DRL)-based optimisation (2, 3). While these methods can encode priorities and constraints explicitly, they generally struggle to integrate Service Level Agreement (SLA) constraints, operator-defined business priority policies and dynamically evolving user behaviour patterns into a unified and adaptive decision-making framework. Rule-based methods generalise poorly to traffic conditions that deviate from those anticipated at design time, whereas DRL methods require extensive training data and careful reward engineering, and provide limited interpretability of their allocation decisions.

The emergent reasoning capabilities of Large Language Models (LLMs) present a potential alternative (4). Since 2023, researchers have begun exploring LLMs as intelligent decision engines for network management. Frameworks such as WirelessAgent (5) have demonstrated that a cognitive architecture combining perception, planning and action, supported by tool calling, can approach rule-optimal performance in slicing scenarios without task-specific training. However, existing LLM-based approaches predominantly adopt a centralised single-agent design in which one model is responsible for resource decisions across all slices, without exposing explicit negotiation among slice interests. This observation motivates the present work.

1.2 Problem Statement

The core problem addressed by this project is the dynamic allocation of shared radio bandwidth among multiple 5G network slices under heterogeneous SLA constraints. Two specific limitations of existing approaches are identified:

1. Existing LLM-based network management frameworks either adopt a centralised single-agent design that does not support explicit inter-slice negotiation (5, 6), or they propose multi-agent architectures whose validation is confined to tasks other than slice resource allocation (7). A complete, end-to-end multi-agent LLM framework dedicated to network slice resource management and validated in a reproducible simulation environment has not yet been demonstrated.

2. General-purpose multi-agent LLM frameworks such as MetaGPT (8) and ChatDev (9) provide rich coordination mechanisms, but these are designed for predominantly cooperative tasks. Network slice resource allocation is better modelled as a constrained multi-party optimisation problem in which the objectives of different slices compete under a fixed capacity budget (10). Structured negotiation and conflict-resolution protocols tailored to this competitive setting remain underexplored.

1.3 Research Objectives

This project aims to design and implement an LLM multi-agent negotiation-based resource allocation framework in a 5G network slicing simulation environment, evaluate it against established baselines, and assess the contribution of the negotiation mechanism. The research is guided by two questions:

RQ1: Can a multi-agent LLM framework match or exceed the performance of traditional rule-based strategies and DRL methods in network slice resource management, as measured by SLA satisfaction rate, resource utilisation and dynamic load adaptability?

RQ2: Does the structured negotiation mechanism between agents provide measurable advantages over a centralised single-round allocation by the coordinator alone, particularly in terms of inter-slice fairness and SLA satisfaction under load conflict scenarios?

1.4 Scope and Assumptions

The following boundaries define the scope of this project:

- The evaluation is conducted entirely in simulation using a custom Gymnasium-based environment. No real network hardware or live traffic traces are used.

- The system considers three slice types (eMBB, URLLC, mMTC) sharing a total bandwidth of 100 MHz, with a maximum of 100 concurrent users.
- SLA definitions are simulation-layer approximations that capture relative priority isolation rather than strict 3GPP end-to-end physical-layer metrics. Specifically, eMBB requires average user throughput ≥ 50 Mbps, URLLC requires 99th-percentile queuing delay $\leq 10\%$ of the eMBB delay, and mMTC requires access success rate $\geq 95\%$.
- The LLM backbone used is a commercial API (GPT-4 series) with temperature fixed at 0 to minimise stochastic variability.
- Each experimental condition is evaluated over 5 independent random seeds across two traffic scenarios (steady-state and burst).

1.5 Contributions

The main contributions of this project are:

1. **A reproducible 5G network slicing simulation environment** built on the Gymnasium framework (11), with clearly defined state space (15 dimensions), action space (3-dimensional continuous allocation), reward function and SLA evaluation logic. All parameters and interfaces are publicly available.
2. **A two-tier scheduling LLM multi-agent decision framework** that reconciles the inference latency of LLMs with the real-time demands of radio resource management. The upper tier features multiple specialised LLM agents — three slice managers and one global coordinator — that collaborate through a structured proposal–evaluation–negotiation protocol. The lower tier executes cached rule-based strategies at millisecond granularity. Two negotiation strategies, Priority Arbitration and Proportional Compromise, are implemented and compared.
3. **A systematic empirical evaluation** comparing six methods (two rule-based baselines, one DRL baseline, one single-agent LLM baseline and two multi-agent LLM variants) together with a negotiation ablation study, across two traffic scenarios and five random seeds per condition.

1.6 Report Organisation

The remainder of this report is structured as follows. Chapter 2 reviews the technical background and related work, covering network slicing architectures, SLA-aware scheduling, LLM-based agent frameworks for network management, multi-agent coordination mechanisms, and the identified research gaps. Chapter 3 presents the design and implementation of the proposed

framework, including the system architecture, two-tier scheduling, agent definitions, negotiation protocol, simulation environment, LangGraph-orchestrated workflow and baseline methods. Chapter 4 describes the experimental setup, presents the results of the two main experiments, and discusses findings, limitations and practical implications. Chapter 5 concludes the report with a summary of achievements, a personal reflection, and suggestions for further work.

Chapter 2: Background

This chapter provides the technical context for the proposed framework. It reviews the principles of network slicing and the associated resource management models, discusses SLA-aware scheduling and two-tier control architectures, surveys the emerging body of work on LLM-based agents for network management, examines general-purpose and network-specific multi-agent coordination mechanisms, and concludes by identifying the research gaps that motivate the design presented in Chapter 3.

2.1 Network Slicing Architecture and Resource Management Models

Network slicing is one of the defining capabilities of 5G and beyond-5G networks. As surveyed by Afolabi *et al.* (1), slicing partitions a shared physical infrastructure into multiple end-to-end logical networks, each tuned to the requirements of a specific service class. The 3GPP service classification distinguishes three canonical slice types — eMBB, URLLC and mMTC — which differ substantially in their throughput, latency, reliability and connection-density targets. eMBB is designed for high-throughput applications such as video streaming and large file downloads, URLLC targets ultra-reliable and low-latency scenarios including industrial control and remote surgery, and mMTC supports massive concurrent device access typical of Internet-of-Things (IoT) sensor deployments. Because these slice types share a common pool of radio resources, resource allocation across slices must simultaneously respect hard capacity constraints and the heterogeneous performance objectives of each tenant.

Two broad families of methods have been applied to this problem. Rule-based and heuristic allocation schemes encode priorities and constraints explicitly and are attractive for their interpretability and bounded computational cost, but they generalise poorly to traffic patterns that deviate from those anticipated at design time. Learning-based approaches, and in particular DRL-based resource managers, have attracted considerable attention in recent years. Li *et al.* (2) formulated bandwidth allocation across slices as a Markov Decision Process (MDP) and demonstrated that DRL agents can outperform hand-crafted heuristics under non-stationary demand. Sciancalepore *et al.* (3) proposed the RL-NSB network slice broker, which uses reinforcement learning to admit and dimension slice requests so as to balance revenue and SLA compliance. Caballero *et al.* (10) analysed slicing under a game-theoretic lens, showing that slice customisation can be supported even when tenants behave strategically, and highlighting the inherently competitive nature of multi-tenant resource sharing. More recently, Nasser and Alani (12) explored hybrid deep-learning architectures for slicing, reinforcing the observation that learning-based methods can adapt to complex workload mixtures but require large amounts of training data and careful reward engineering.

A common limitation across these approaches is that the decision-making component is mono-

lithic: a single policy is trained or hand-crafted to output allocations for all slices simultaneously. Neither the explicit articulation of the individual objectives of each slice nor a structured mechanism for reconciling conflicts between them is provided by the architecture itself. This limitation becomes particularly significant when slice demands change rapidly or when multiple SLA constraints must be balanced under tight resource budgets.

2.2 SLA-Aware Scheduling and Two-Tier Control Mechanisms

Slice resource management must operate simultaneously at two very different timescales. At the radio interface, scheduling decisions are taken on a per-Transmission Time Interval (TTI) basis — typically every one or two milliseconds — and must account for instantaneous channel quality, buffer occupancy and user fairness. At a longer timescale, inter-slice resource quotas, admission control decisions and policy updates are driven by aggregated performance indicators such as SLA satisfaction rates, tenant-level throughput and long-term utilisation.

This separation has led to the adoption of two-tier or hierarchical control architectures in which fast rule-based or model-based controllers handle per-TTI decisions, while slower optimisation or learning-based components adjust the parameters of the lower tier. Such architectures are compatible with the hierarchy already present in modern radio access networks, in which Medium Access Control (MAC)-layer schedulers operate at millisecond granularity while Radio Resource Management (RRM) functions revise resource partitions on a coarser timescale.

In the context of LLM-based decision making, the two-tier view is particularly relevant because the inference latency of contemporary LLMs is typically on the order of hundreds of milliseconds to several seconds per call, which renders it infeasible to place an LLM directly on the per-TTI control loop. Recent work on LLM-augmented O-RAN control (13) and on dual-loop LLM-based management (14) has therefore advocated the separation of a slow semantic reasoning loop from a fast execution loop, foreshadowing the two-tier design adopted in this project. The key engineering challenge lies in determining an appropriate decision frequency for the LLM layer, establishing a reliable fallback mechanism when inference latency exceeds acceptable thresholds, and ensuring that the lower tier can execute coherent allocation policies between successive upper-tier updates.

2.3 LLM-Based Agent Frameworks in Network Management

Between 2023 and 2025, research on LLMs for wireless networks developed along three broadly distinct technical routes, each making different trade-offs in how domain knowledge is injected into the model.

2.3.1 Prompt Engineering and Agent Workflow Route

The first route relies on prompt engineering and explicit agent workflows. WirelessAgent (5) decomposes network management into a sequential workflow of intent understanding, slice allocation, bandwidth allocation and load balancing, where each stage is implemented as a LangGraph node that manages global state transitions. Its main contribution is demonstrating that a cognitive architecture consisting of perception, memory, planning and action, combined with tool calling, can approach rule-optimal performance in slicing scenarios without any task-specific training — the reported bandwidth utilisation is only 4.3% below the rule-optimal baseline under its considered experimental configuration, and 44.4% higher than pure prompting methods. WirelessLLM (6) proposes a complementary methodological roadmap: rapid domain knowledge injection through prompt engineering and Retrieval-Augmented Generation (RAG) (15) (knowledge alignment), fusion with specialised models (knowledge fusion), and model evolution through continual learning. Together, these works suggest that prompt-based approaches can be effective for network management when combined with appropriate architectural scaffolding, though they do not address the multi-agent coordination problem.

2.3.2 Parameter Fine-Tuning Route

The second route addresses domain adaptation through parameter fine-tuning. NetLLM (16) reformulates a selection of networking problems — including viewport prediction, adaptive bitrate selection and cluster job scheduling — as sequential decision problems and fine-tunes a Llama2-based backbone using low-rank adaptation (LoRA), reporting improvements over DRL baselines on all three tasks. While effective, this route requires substantial labelled data for each target task, a condition that is not easily satisfied in the network slicing domain where annotated operational datasets are scarce. Furthermore, fine-tuned models may lose their general reasoning capabilities, potentially limiting their applicability to novel or unforeseen network conditions.

2.3.3 Cloud-Edge Collaboration Route

The third route explores cloud-edge collaboration architectures. NetGPT (17) proposes an architecture in which lightweight LLMs are deployed at the edge for latency-sensitive inference while larger models remain in the cloud for global optimisation, anticipating the deployment challenges posed by the size and inference cost of frontier models. Related work by Chergui *et al.* (18) has investigated collective memory and knowledge sharing across LLM-based agents for cross-domain 6G management, exploring how distributed LLM agents can maintain coherent decision-making across heterogeneous network domains.

2.3.4 Summary

A comprehensive survey by Zhou *et al.* (4) provides a broader overview of LLM applications across the telecommunications sector, documenting the rapid growth of this field and the diversity of approaches being explored. Across all three routes identified above, the decision-making agent is typically centralised: a single LLM or LLM-powered component is responsible for the entire resource management task. This observation underlies one of the research gaps identified later in this chapter.

2.4 Multi-Agent Coordination and Negotiation Mechanisms

2.4.1 General-Purpose Multi-Agent LLM Frameworks

In parallel with the network-specific work reviewed above, general-purpose multi-agent LLM systems have become an active area of research. MetaGPT (8) encodes Standard Operating Procedures (SOPs) as prompt sequences and enforces the exchange of structured intermediate artefacts between agents, reducing the information loss that can arise when free-form natural language is the sole communication channel. AutoGen (19) provides a flexible conversational programming paradigm in which agents with different roles coordinate through multi-turn dialogues, and has been applied to tasks including code generation and data analysis. CAMEL (20) investigates the emergence of cooperative behaviour in role-playing agent societies, demonstrating that LLM agents can develop task-specific collaborative strategies through structured interaction. ChatDev (9) applies multi-agent collaboration to software development by assigning dedicated roles to designers, coders and testers, showcasing the benefits of role specialisation in complex tasks. A survey by Wang *et al.* (21) provides a broader taxonomy of LLM-based autonomous agents and their cognitive components, identifying perception, planning, action and memory as the four foundational modules.

These frameworks have matured rapidly, but they share an important assumption: the underlying task is predominantly cooperative and admits a natural division of labour. Adapting them to network resource management raises three specific difficulties:

1. Slice resource allocation is a *competitive* constrained optimisation problem in which the total bandwidth is fixed and agents representing different slices must compete for a limited pool of resources.

2. The decisions are subject to strong real-time constraints, which complicates the use of unbounded multi-turn dialogues.

3. Decision outcomes are associated with clear quantitative evaluation criteria — SLA satisfaction rates, throughput, latency — which leave limited room for the qualitative judge-

ment that dominates many general-purpose multi-agent benchmarks.

2.4.2 Multi-Agent Approaches in the Networking Domain

Multi-agent attempts that target networking tasks directly remain limited. CommLLM (7) proposes a three-component multi-agent architecture for 6G communications consisting of retrieval, cooperative planning and evaluation modules, but the reported validation is primarily focused on semantic communication and does not constitute a full slicing evaluation with quantitative resource allocation metrics. The dual-loop edge-terminal collaboration of Qu *et al.* (14) is architecturally informative in its treatment of an LLM-driven semantic loop coupled with a fast execution loop, yet coordination within that framework remains predominantly centralised, with limited inter-agent negotiation. ORAN-GUIDE (13) adopts a division of labour in which an LLM provides semantic understanding and a reinforcement learning component handles online optimisation, offering useful insight into how LLM decision latency can be decoupled from network real-time requirements, but it does not implement multi-agent collaboration in the sense of multiple LLMs representing different network entities.

2.4.3 Relevant Techniques

Two techniques commonly adopted within these frameworks deserve specific mention. Chain-of-Thought (CoT) prompting (22) encourages the model to reason step by step, and has been shown to improve performance on tasks involving arithmetic or logical decomposition; it is relevant for the structured resource allocation reasoning required of slice-manager agents. RAG (15) is frequently adopted as a lightweight mechanism for injecting domain knowledge without modifying model parameters, and can be used to provide agents with up-to-date network configuration data or SLA specifications.

2.5 Identified Research Gaps

Taken together, the work reviewed above suggests two complementary opportunities that this project seeks to address.

Gap 1: Absence of an end-to-end multi-agent LLM framework for slice resource management. Existing LLM-based network management frameworks either adopt a centralised single-agent design and do not expose explicit negotiation between slices (5, 6), or they propose multi-agent architectures whose validation is confined to communication tasks other than slice resource allocation (7). A complete framework dedicated to slice resource management that supports explicit multi-agent collaboration and is validated end-to-end in a reproducible slicing environment has not yet been presented in the accessible literature.

Gap 2: Lack of structured negotiation protocols for competitive resource allocation. Although general-purpose multi-agent LLM frameworks such as MetaGPT (8) and ChatDev (9) provide rich coordination mechanisms, these are designed with cooperative tasks in mind. Slice allocation is better modelled as a constrained multi-party optimisation problem in which the objectives of different slices compete under a fixed capacity budget (10). Structured negotiation and conflict-resolution protocols tailored to this competitive setting have received limited attention in the network management literature.

Table 1 summarises the positioning of this work relative to related frameworks across five key dimensions.

Table 1: Comparison of related frameworks and positioning of this research.

Feature	**WirelessAgent** (5)	**CommLLM** (7)	**Qu et al.** (14)	**This Research**
Agent count	Single agent	Multi-agent (concept)	Dual-loop multi-agent	Multi-agent (3 slice + 1 coordinator)
Validation scenario	Network slicing	Semantic comm.	Edge-terminal collab.	Network slice resource allocation
Negotiation mechanism	None	Unspecified	Centralised coord.	Proposal–evaluation–negotiation protocol
Real-time handling	No explicit constraint	Not discussed	Inner-loop replanning	Two-tier scheduling (LLM low-freq + rule high-freq)
Simulation validation	Custom env.	Concept level	Partial validation	Gymnasium standardised env.

The design described in the following chapter responds to these two gaps by combining a two-tier scheduling architecture with a set of specialised slice-manager agents and a global coordinator that interact through an explicit proposal–evaluation–negotiation protocol.

Chapter 3: Design and Implementation

This chapter presents the design of the proposed LLM multi-agent negotiation framework for 5G network slice resource management and describes its implementation. Section 3.1 outlines the overall system architecture and the guiding design principles. Section 3.2 details the two-tier scheduling framework. Section 3.3 specifies the roles and internal logic of the individual agents. Section 3.4 defines the proposal–evaluation–negotiation protocol. Section 3.5 describes the Gymnasium-based simulation environment. Section 3.6 discusses the LangGraph-orchestrated workflow. Section 3.7 presents the baseline methods implemented for comparison.

3.1 System Overview and Architectural Principles

The proposed framework is organised into three loosely coupled layers, as illustrated in Figure 1. The *Agent Layer* defines the specialised agents and their configurations. The *Collaboration Layer* implements inter-agent negotiation and conflict resolution. The *Environment Layer* provides a standardised simulation interface that decouples the decision logic from the underlying simulator. The full system is orchestrated through LangGraph (23), with a commercial LLM API (GPT-4 series) serving as the reasoning backbone.

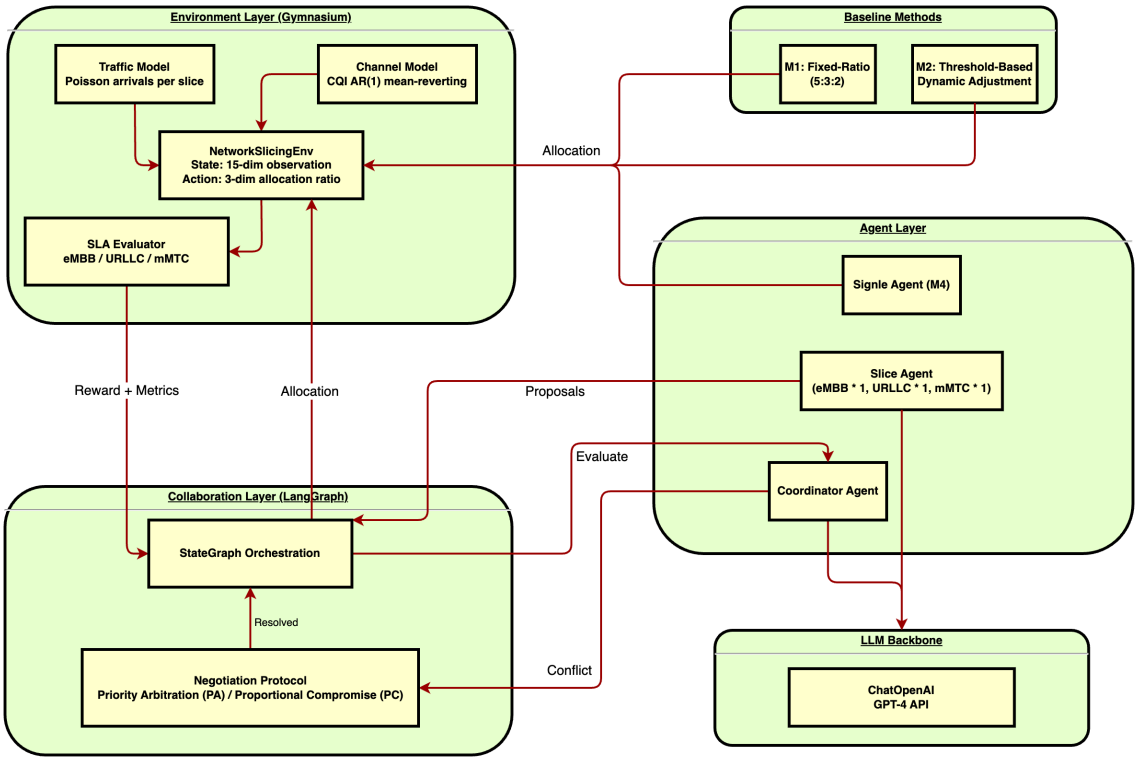


Figure 1: Overall system architecture of the proposed two-tier multi-agent framework.

The design is guided by four principles:

- *Separation of timescales*: Slow semantic reasoning by LLM agents is decoupled from fast

rule-based execution so that LLM inference latency does not propagate into the per-TTI control loop.

- *Role specialisation*: Each slice type is represented by a dedicated agent whose prompts and internal reasoning are tailored to the characteristics of that slice, while a global coordinator agent is responsible exclusively for cross-slice arbitration.
- *Structured communication*: Inter-agent messages are exchanged as structured JSON artefacts rather than as free-form natural language, in the spirit of MetaGPT (8), in order to reduce information loss and to make the behaviour of the system inspectable.
- *Graceful degradation*: Whenever the LLM layer fails to produce a decision within a predefined latency budget, the system automatically falls back to a rule-based controller, so that the operational stability of the network is never contingent on the responsiveness of an external API.

3.2 Two-Tier Scheduling Framework

A central challenge addressed by the design is the order-of-magnitude gap between the inference latency of contemporary LLMs — typically hundreds of milliseconds to a few seconds per call — and the millisecond-level timescale at which radio scheduling decisions must be taken. Placing an LLM directly on the per-TTI control loop is therefore infeasible. The proposed framework adopts an explicit two-tier scheduling architecture, illustrated in Figure 2.

The *upper tier*, or LLM decision layer, is triggered once every 100 TTIs (approximately one second) in the baseline configuration. At each trigger, the slice-manager agents and the global coordinator agent execute one round of the proposal–evaluation–negotiation protocol described in Section 3.4 and update the inter-slice bandwidth allocation quotas. An emergency decision can additionally be triggered when the measured SLA violation rate exceeds a predefined threshold, allowing the framework to react to sudden changes in traffic conditions.

The *lower tier*, or fast execution layer, operates at a TTI-level granularity of approximately 10 ms. Within each decision window, it executes a cached rule-based strategy that enforces the inter-slice quotas produced by the upper tier, without invoking the LLM. This layer is responsible for per-user scheduling inside each slice and for monitoring SLA-related indicators that feed back into the upper tier.

A *maximum decision latency threshold* of 2 seconds is enforced on the upper tier. If the LLM layer fails to return a decision within this budget, the system automatically falls back to the most recent valid allocation or, in the absence of one, to a default fixed-ratio policy (eMBB : URLLC : mMTC $= 0.5 : 0.3 : 0.2$). This fallback behaviour is a direct consequence of the graceful degradation principle introduced in Section 3.1.

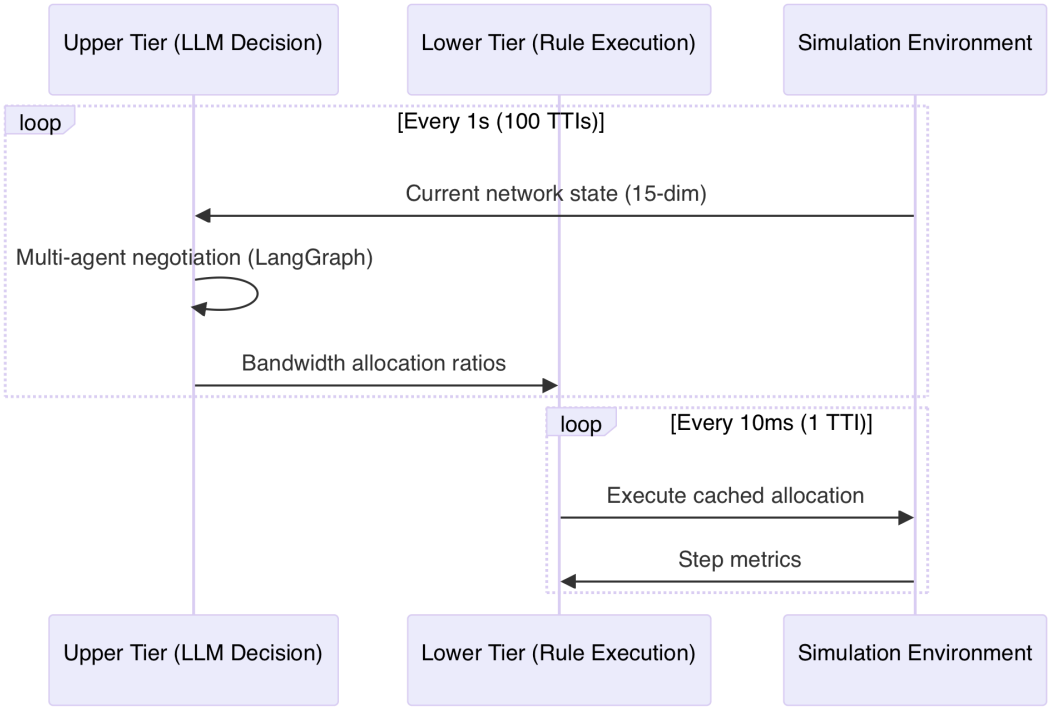


Figure 2: Two-tier scheduling: slow LLM policy updates at the upper tier and fast rule-based execution at the lower tier.

3.3 Agent Role Definition and Internal Logic

The proposed framework defines four agents: one slice-manager agent per slice type (eMBB, URLLC, mMTC) and a single global coordinator agent. Each agent shares a common internal structure consisting of three modules — *Perception*, *Planning* and *Action* — in line with the cognitive architecture pattern adopted by WirelessAgent (5) and surveyed more broadly by Wang *et al.* (21).

Slice Manager Agent. A slice-manager agent is instantiated for each of the three slice types. The Perception module converts the numerical state of the slice — user count, average Channel Quality Indicator (CQI), buffer occupancy and current SLA satisfaction rate — into a structured natural-language description that is fed into the LLM prompt. The Planning module uses Chain-of-Thought prompting (22) to assess the current resource demand of the slice in light of its SLA targets. The Action module outputs a resource demand proposal as a structured JSON object containing four fields:

- `requested`: the desired bandwidth fraction;
- `minimum`: the minimum acceptable fraction (always ≥ 0.05);
- `priority`: a declaration of “high”, “medium” or “low”;

- `justification`: a brief natural-language rationale.

Each agent carries a role-specific system prompt that encodes the SLA requirements and operational characteristics of its assigned slice. When the coordinator provides feedback after a failed compatibility check, the slice agent can generate a revised *counter-proposal* that takes the feedback into account while still advocating for its slice's interests.

Global Coordinator Agent. A single global coordinator agent is responsible for cross-slice arbitration. Its Perception module aggregates the proposals produced by the slice-manager agents together with the global resource state. The Planning module determines whether the proposals are jointly compatible — that is, whether the sum of the requested shares can be satisfied within the available bandwidth budget. If the proposals are compatible, they are approved directly. If a conflict is detected, the coordinator outputs a structured evaluation as a JSON object containing:

- `compatible`: a boolean indicating whether proposals are jointly feasible;
- `allocation`: a dictionary mapping each slice name to its allocated fraction;
- `feedback`: a natural-language rationale or per-slice suggestions.

The priority order URLLC $>$ eMBB $>$ mMTC is encoded in the coordinator's system prompt. The allocation produced by the coordinator is always renormalised to sum to 1, with each slice receiving at least 0.05.

3.4 Proposal–Evaluation–Negotiation Protocol

When the coordinator detects that the aggregate demand exceeds the available capacity, a three-phase negotiation protocol is triggered. The overall message flow is shown in Figure 3.

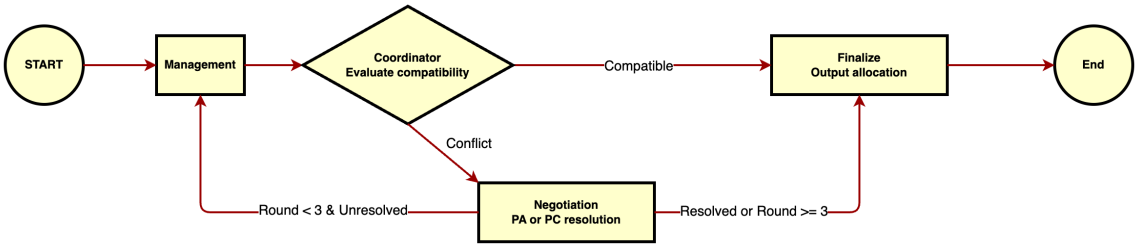


Figure 3: Message flow of the three-phase proposal–evaluation–negotiation protocol.

Phase 1 — Proposal. Each slice-manager agent independently evaluates its current demand and submits a resource proposal as a structured JSON message containing its requested share, minimum acceptable share, priority declaration and justification.

Phase 2 — Evaluation. The global coordinator agent evaluates the joint feasibility of the proposals. Feasibility checks include whether the total requested bandwidth exceeds unity and whether hard priority rules are satisfied (e.g., URLLC requirements take precedence).

Phase 3 — Negotiation. If a conflict is detected, a deterministic resolution protocol is invoked. The implementation provides two strategies:

- **Priority Arbitration (PA):** Each slice receives its minimum allocation of 0.05. The remaining capacity is distributed in priority order (URLLC $>$ eMBB $>$ mMTC), with each slice receiving up to its requested share before the next slice is considered.
- **Proportional Compromise (PC):** Each slice is first guaranteed its declared minimum. The residual capacity ($1.0 - \sum$minimums) is then distributed in proportion to the excess demand (requested $-$ minimum) of each slice.

After a resolution round, the graph may loop back to the slice-agent node for a further round of counter-proposals, up to a maximum of 3 negotiation rounds. If the maximum is reached without the coordinator declaring compatibility, the most recent resolution is accepted as the final allocation. This bounded iteration controls both latency and token cost.

3.5 Gymnasium-Based Simulation Environment

A custom 5G network slicing simulation environment is built using the Gymnasium framework (11). The environment models the dynamics of user arrivals, channel quality variations, buffer accumulation and SLA evaluation at each decision step. All parameters are listed in Table 2.

3.5.1 State Space

The observation space comprises 15 continuous dimensions, each normalised to $[0, 1]$, as defined in Table 3.

3.5.2 Action Space

The action space is a continuous 3-dimensional vector $\mathbf{a} = (a_{\text{eMBB}}, a_{\text{URLLC}}, a_{\text{mMTC}})$ subject to $\sum_i a_i = 1$ and $a_i \geq 0.05$. Raw actions are clipped to $[0, 1]$, the minimum allocation constraint is enforced, and the result is renormalised to ensure the fractions sum to unity.

Table 2: Key parameters of the simulation environment.

Parameter	**Value**	**Notes**
System bandwidth	100 MHz	Reference NR n78 band
Slice types	eMBB, URLLC, mMTC	3GPP canonical types
Maximum users	100	Per decision cycle
Poisson arrival rates	[50, 30, 20]	eMBB, URLLC, mMTC
CQI means	[9, 10, 7]	eMBB, URLLC, mMTC
CQI reversion rate (α)	0.15	AR(1) mean reversion
CQI noise std (σ)	1.2	Gaussian
Minimum allocation	0.05	Per slice
Burst onset cycle	50	eMBB λ doubles
Burst multiplier	2.0	eMBB arrival rate
SLA sliding window	10 steps	For satisfaction rate
Decision cycles per episode	100	`max_steps`

3.5.3 Channel and Throughput Model

Channel quality is modelled using a discrete Ornstein–Uhlenbeck (AR(1)) process for each slice:

$$\mathrm{CQI}_i(t+1) = \mathrm{CQI}_i(t) + \alpha\big(\mu_i - \mathrm{CQI}_i(t)\big) + \sigma\,\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0,1) \tag{1}$$

where μ_i is the long-term mean CQI for slice i, α is the reversion rate and σ is the noise standard deviation. CQI values are rounded to integers and clipped to $[1, 15]$. Spectral efficiency is obtained from the standard 3GPP CQI-to-spectral-efficiency mapping table. User-level throughput is computed as:

$$T_{i,\mathrm{user}} = \frac{a_i \cdot B_{\mathrm{total}} \cdot \mathrm{SE}(\mathrm{CQI}_i) \cdot M}{n_i} \tag{2}$$

where $B_{\mathrm{total}} = 100\,\mathrm{MHz}$ is the total bandwidth, $\mathrm{SE}(\cdot)$ is the spectral efficiency function, $M = 20$ is a MIMO and coding gain multiplier, and n_i is the number of users in slice i.

3.5.4 SLA Definitions

The SLA metrics used in this simulation represent relative priority isolation rather than strict 3GPP end-to-end physical-layer metrics. Table 4 defines the three SLA conditions.

Table 3: State space definition and normalisation.

Dimension	**Meaning**	**Range**	**Normalisation**
n_i ($\times 3$)	Current user count per slice	$[0, N_{\max}]$	$/\, N_{\max}$
$\bar{c}_i$ ($\times 3$)	Average CQI per slice	$[1, 15]$	$/\, 15$
b_i ($\times 3$)	Buffer occupancy per slice	$[0, 1]$	Raw value
a_i ($\times 3$)	Current bandwidth allocation	$[0, 1]$	Raw value
s_i ($\times 3$)	SLA satisfaction rate (sliding window)	$[0, 1]$	Raw value

Table 4: Simulation-layer SLA approximations and design intent.

Slice	**SLA Condition**	**Design Intent**	**Application**
eMBB	Average user throughput ≥ 50 Mbps	High throughput guarantee	Video streaming
URLLC	99th-percentile queuing delay $\leq$ 10% of eMBB delay	Relative priority guarantee	Industrial control
mMTC	Access success rate $\geq 95\%$	Massive access guarantee	IoT sensors

For URLLC, the 99th-percentile delay is approximated as 2.3 times the mean queuing delay (based on an exponential service model), and the delay is computed as a packet-weight-to-capacity ratio. The mMTC access rate is modelled as the ratio of allocated capacity to the number of requesting devices.

3.5.5 Reward Function

The reward at each step is a weighted combination of three normalised terms:

$$R = w_S \cdot \bar{S} + w_U \cdot U - w_V \cdot V \tag{3}$$

where $\bar{S}$ is the mean SLA satisfaction rate across slices (computed over a sliding window of 10 steps), U is the bandwidth utilisation, and $V = 1 - \bar{S}_{\text{instant}}$ is the instantaneous violation penalty. The default weights are $w_S = 0.5$, $w_U = 0.3$ and $w_V = 0.2$.

3.5.6 Traffic Scenarios

Two traffic scenarios are used throughout the experiments:

- **Steady-state:** User arrival rates follow Poisson processes with fixed means (eMBB: 50, URLLC: 30, mMTC: 20) throughout the episode.
- **Burst:** Identical to steady-state for the first 49 decision cycles; from cycle 50 onward, the eMBB arrival rate is doubled ($\lambda_{\text{eMBB}} = 100$), simulating a sudden surge in mobile broadband demand.

3.6 LangGraph-Orchestrated Agent Workflow

The multi-agent decision process is orchestrated using LangGraph (23), a state-machine library that represents the agent workflow as a directed graph with typed state transitions. The graph carries a shared state object (`MASState`) through four node types:

1. **Slice Agents node:** Invokes all three slice-manager agents in sequence, each producing a proposal (or counter-proposal if feedback is available from a prior round).
2. **Coordinator node:** Invokes the coordinator agent to evaluate the joint feasibility of the proposals and produce an allocation.
3. **Negotiation node:** If the coordinator declares the proposals incompatible, a deterministic resolution function (PA or PC) computes an allocation from the proposals.
4. **Finalize node:** Extracts the final allocation from either the coordinator's approved allocation or the negotiation resolution.

Conditional edges connect the coordinator node to either the finalize node (if proposals are compatible) or the negotiation node (if a conflict is detected). After negotiation, the graph may loop back to the slice-agents node for another round, up to the configured maximum of 3 rounds.

The no-negotiation variant (M5-NoNeg) uses a simplified graph in which the coordinator node connects directly to the finalize node, with no negotiation or loopback edges. In this variant, the coordinator's single-round allocation is used as the final decision.

3.7 Baseline Methods and Implementation Details

Six methods are implemented and compared, as summarised in Table 5.

Rule-Based Baselines (M1, M2). M1 applies a fixed allocation ratio throughout the episode, following commonly adopted traffic composition settings (12). M2 monitors the sliding-window SLA satisfaction rate for each slice; when any slice falls below a threshold of 0.8, it transfers a small amount of bandwidth (0.05) from the slice with the highest current SLA rate, subject to the minimum allocation constraint.

Table 5: Summary of compared methods (M1–M5 and variants).

ID	Method	Category	Description
M1	Fixed-Ratio	Rule-based	Static allocation (eMBB : URLLC : mMTC $= 0.5 : 0.3 : 0.2$)
M2	Threshold-based	Rule-based	Dynamic adjustment triggered when any slice's SLA satisfaction rate drops below 0.8; bandwidth is transferred from the best-performing slice
M3	PPO	Deep RL	MLP policy network trained with Proximal Policy Optimisation (24) using Stable-Baselines3 (25)
M4	Single-Agent LLM	Single-agent LLM	One GPT-4 agent with CoT reasoning; no multi-agent collaboration
M5-PA	Multi-Agent (PA)	Multi-agent LLM	Slice agents + coordinator + Priority Arbitration negotiation
M5-PC	Multi-Agent (PC)	Multi-agent LLM	Slice agents + coordinator + Proportional Compromise negotiation

DRL Baseline (M3). The PPO agent (24) is implemented using Stable-Baselines3 (25) with an MLP policy network. Training uses a mixed-scenario wrapper that randomly selects between steady-state and burst scenarios at each episode reset (burst probability 0.4). Training hyperparameters include learning rate 3×10^{-4}, 1024 steps per update, batch size 64, 10 epochs per update, and discount factor $\gamma = 0.99$. Early stopping is applied when the mean reward change over the most recent 100 episodes falls below 0.5%, with a maximum of 2000 training episodes.

Single-Agent LLM Baseline (M4). A single GPT-4 agent receives the full environment state as a natural-language description and outputs a 3-dimensional allocation vector via Chain-of-Thought reasoning. No inter-agent collaboration or negotiation takes place.

Multi-Agent LLM (M5). The full multi-agent framework described in Sections 3.1–3.6, tested with both PA and PC negotiation strategies. The no-negotiation ablation (M5-NoNeg) retains the three slice agents and the coordinator but removes the negotiation loop, so that the coordinator's initial allocation is used directly.

Chapter 4: Results and Discussion

This chapter presents and analyses the experimental results. Section 4.1 describes the experimental design and evaluation metrics. Section 4.2 reports the core performance comparison across all six methods (Experiment 1, addressing RQ1). Section 4.3 presents the negotiation mechanism validation (Experiment 2, addressing RQ2). Section 4.4 provides a consolidated analysis of the results. Section 4.5 discusses threats to validity and limitations. Section 4.6 considers practical implications.

4.1 Experimental Design and Evaluation Metrics

4.1.1 Experimental Configuration

All experiments share a common configuration. The simulation environment uses 100 MHz total bandwidth with three slice types (eMBB, URLLC, mMTC). Each episode runs for 100 decision cycles. Five random seeds (42, 123, 456, 789, 1024) are used for each method–scenario combination, and two traffic scenarios (steady-state and burst) are evaluated. The LLM temperature is fixed at 0 to limit variability to the environment's stochastic dynamics. Table 6 summarises the full experimental configuration.

Table 6: Experimental configuration.

Parameter	**Setting**
System bandwidth	100 MHz
Slice types	eMBB / URLLC / mMTC
User scale	Up to 100 users per cycle
Decision cycles per episode	100
Random seeds	5 (42, 123, 456, 789, 1024)
Traffic scenarios	Steady-state; Burst (eMBB λ doubles at cycle 50)
LLM temperature	0
Maximum negotiation rounds	3

4.1.2 Evaluation Metrics

Six metrics are used to evaluate each method, as defined in Table 7.

Table 7: Evaluation metrics and computation definitions.

Metric	**Computation**
Per-slice SLA satisfaction rate	Proportion of decision cycles in which the slice's SLA condition is met, averaged over the episode
Weighted total throughput	Sum of all user throughputs across slices (Mbps)
Bandwidth utilisation	Ratio of actual demand-driven load to total system bandwidth
Inter-slice fairness	Jain's fairness index (26) computed on the per-slice SLA satisfaction rates
Per-decision latency	Wall-clock time from state input to allocation output (seconds)
Token consumption	Total tokens consumed per episode (input + output), for LLM-based methods

4.1.3 Experiment Design

Experiment 1 (Core Performance Comparison — RQ1): Methods M1 through M5-PC are run under both steady-state and burst scenarios with 5 seeds each, yielding 60 episode-level data points. The aim is to quantify the performance differences between the multi-agent LLM framework and established baselines.

Experiment 2 (Negotiation Mechanism Validation — RQ2): The three multi-agent variants — M5-PA, M5-PC and M5-NoNeg (no negotiation) — are compared under both scenarios with 5 seeds each, yielding 30 episode-level data points. The aim is to isolate the contribution of the negotiation mechanism from the effect of additional LLM computation.

4.2 Core Performance Comparison (Experiment 1)

4.2.1 Overall Performance

Table 8 presents the mean and standard deviation of the key metrics across all methods and scenarios. The results reveal several noteworthy patterns.

4.2.2 Per-Slice SLA Satisfaction

Figure 4 presents the per-slice SLA satisfaction rates for each method under both scenarios. The results show distinct allocation strategies across methods.

Table 8: Experiment 1: Core performance comparison (mean ± std over 5 seeds).

Method	Scenario	SLA Avg	Throughput (Mbps)	BW Util	Fairness	Latency (s)	Tokens/ep
M1-Fixed	burst	.612±.029	4769±218	.760±.006	.819±.067	0	0
M1-Fixed	steady	.617±.040	4617±254	.538±.010	.899±.072	0	0
M2-Thres	burst	.399±.027	4467±325	.760±.006	.923±.034	0	0
M2-Thres	steady	.491±.020	4379±212	.538±.010	.956±.023	0	0
M3-PPO	burst	.515±.027	4651±222	.760±.006	.890±.033	<0.001	0
M3-PPO	steady	.499±.015	4350±262	.538±.010	.950±.035	<0.001	0
M4-Single	burst	.481±.037	5074±267	.760±.006	.645±.071	5.79±2.46	26244
M4-Single	steady	.500±.036	4940±332	.538±.010	.726±.036	8.87±0.24	26124
M5-PA	burst	.389±.029	5549±365	.760±.006	.513±.060	6.65±1.58	124838
M5-PA	steady	.413±.022	5336±404	.538±.010	.613±.048	8.07±0.11	124938
M5-PC	burst	.606±.021	4650±176	.760±.006	.794±.055	6.63±1.48	125348
M5-PC	steady	.585±.037	4635±232	.538±.010	.865±.065	7.94±0.19	125244

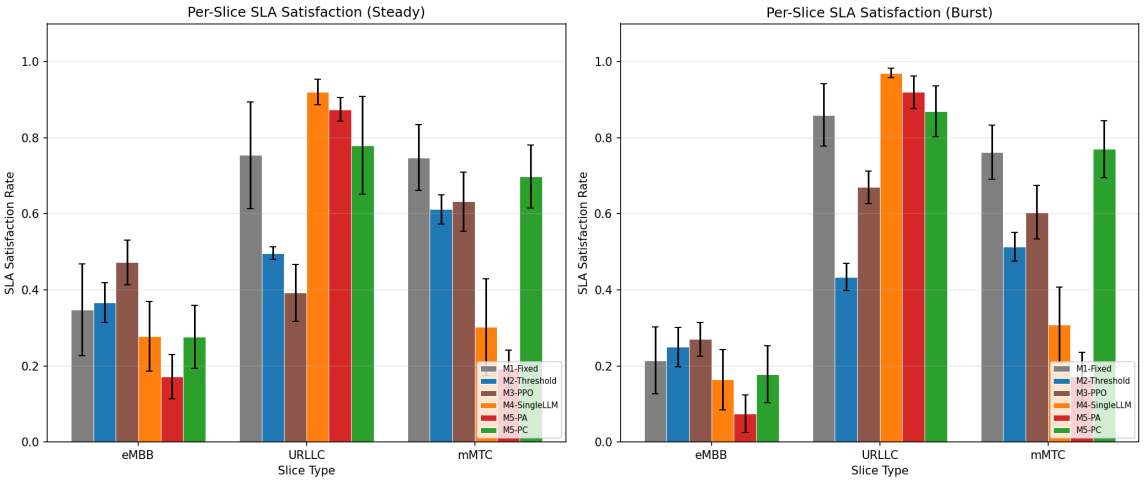


Figure 4: Experiment 1: Per-slice SLA satisfaction rates by method and scenario.

The LLM-based methods (M4, M5-PA) consistently achieve high URLLC SLA satisfaction (0.87–0.97 across scenarios), reflecting the priority encoding in their system prompts. However, this comes at the cost of lower eMBB and mMTC satisfaction, particularly for M5-PA which achieves only 0.074 eMBB satisfaction under burst conditions. By contrast, M5-PC achieves a more balanced distribution across slices (eMBB: 0.178, URLLC: 0.870, mMTC: 0.770 under burst), comparable to the rule-based M1-Fixed baseline.

M1-Fixed achieves the highest average SLA (0.612 burst, 0.617 steady) despite its simplicity, because its fixed 5:3:2 ratio happens to allocate a relatively generous share to eMBB and mMTC, which are the most bandwidth-demanding slices in the simulation. M2-Threshold, while achieving the highest fairness scores (0.923 burst, 0.956 steady), suffers from lower overall SLA satisfaction because its reactive bandwidth transfers can overshoot, destabilising allocations under dynamic conditions.

4.2.3 Burst Scenario Adaptation

Figure 5 shows the time-series evolution of the average SLA satisfaction rate under the burst scenario, with the burst event occurring at cycle 50.

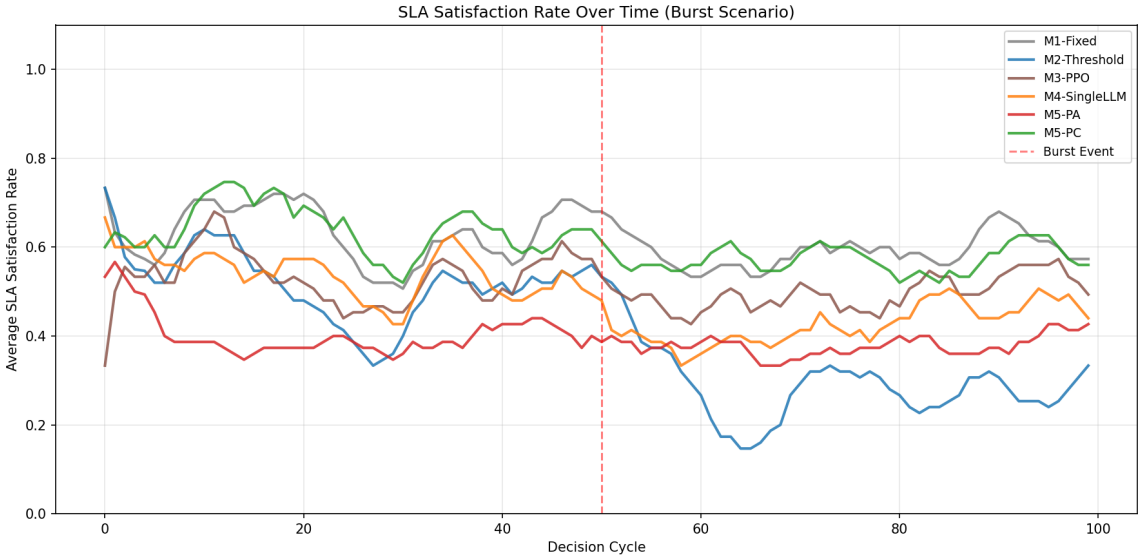


Figure 5: Experiment 1: Average SLA satisfaction trend under burst scenario (burst onset at cycle 50, marked by dashed line).

Following the burst onset, all methods experience a decline in average SLA satisfaction, reflecting the increased pressure on eMBB resources. M1-Fixed and M5-PC show relatively stable post-burst behaviour, while M2-Threshold exhibits more oscillatory responses as its reactive mechanism attempts to track the new demand pattern. The LLM-based methods (M4, M5) show varying degrees of adaptation, with their responses shaped by the reasoning encoded in their prompts rather than by learned or hard-coded rules.

4.2.4 Performance Overview

Figure 6 provides a comprehensive view of additional metrics including throughput, bandwidth utilisation, fairness, decision latency and token consumption.

Bandwidth utilisation is effectively identical across all methods within each scenario (0.760 for burst, 0.538 for steady), as this metric is determined by the user demand profile rather than the allocation strategy. Throughput varies more substantially: M5-PA achieves the highest throughput (5549 Mbps under burst) due to its aggressive allocation towards high-priority slices, while M2-Threshold produces the lowest (4467 Mbps under burst). The LLM-based methods incur substantial decision latencies (5.8–8.9 seconds per step), compared to effectively zero for rule-based methods and negligible latency for PPO. Token consumption is approximately 26,000 per episode for M4 and 125,000 for M5 variants, reflecting the additional LLM calls in the multi-

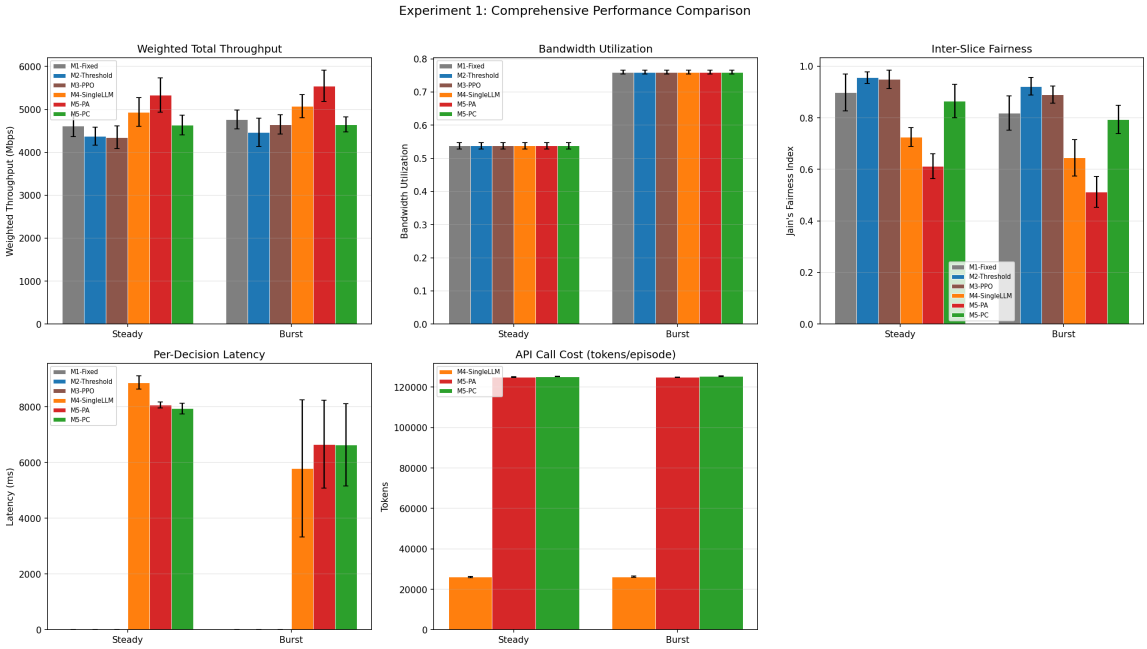


Figure 6: Experiment 1: Performance overview across metrics and scenarios.

agent framework.

4.3 Negotiation Mechanism Validation (Experiment 2)

4.3.1 Comparison of Multi-Agent Variants

Table 9 compares the three multi-agent variants: M5-PA (Priority Arbitration), M5-PC (Proportional Compromise) and M5-NoNeg (no negotiation, coordinator-only allocation).

Table 9: Experiment 2: Multi-agent variant comparison (mean ± std over 5 seeds).

Method	Scenario	SLA Avg	Throughput (Mbps)	BW Util	Fairness	Latency (s)	Tokens/ep
M5-NoNeg	burst	.481±.046	4855±252	.760±.006	.780±.104	6.51±1.47	125398
M5-NoNeg	steady	.463±.079	4840±320	.538±.010	.837±.021	8.14±0.38	125512
M5-PA	burst	.389±.029	5549±365	.760±.006	.513±.060	6.65±1.58	124838
M5-PA	steady	.413±.022	5336±404	.538±.010	.613±.048	8.07±0.11	124938
M5-PC	burst	.606±.021	4650±176	.760±.006	.794±.055	6.63±1.48	125348
M5-PC	steady	.585±.037	4635±232	.538±.010	.865±.065	7.94±0.19	125244

4.3.2 Fairness Comparison

Figure 7 compares the Jain's fairness index across the three variants. M5-PC consistently achieves higher fairness than M5-PA, with the gap being more pronounced under burst conditions (0.794

vs. 0.513). M5-NoNeg falls between the two negotiation strategies in terms of fairness (0.780 under burst), though with considerably higher variance (±0.104).

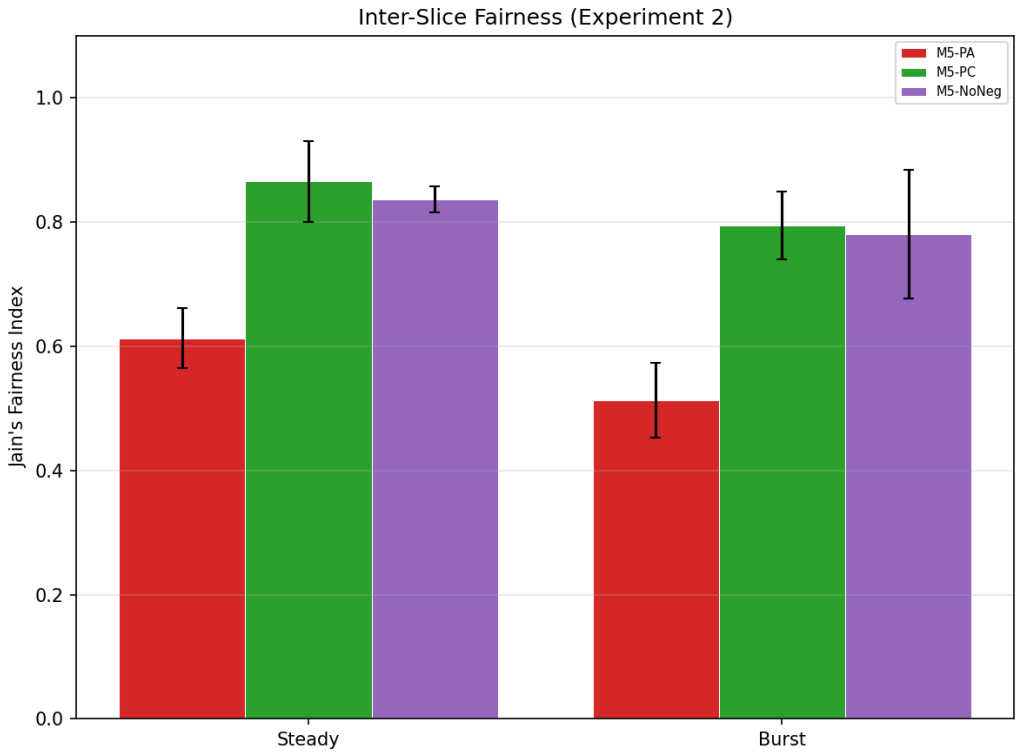


Figure 7: Experiment 2: Jain's fairness index comparison across multi-agent variants.

4.3.3 Per-Slice SLA Analysis

Figure 8 shows the per-slice SLA satisfaction rates for the three variants, revealing the mechanisms by which each achieves its aggregate performance.

M5-PA achieves the highest URLLC satisfaction (0.920 under burst) but the lowest mMTC satisfaction (0.172), reflecting the strict priority ordering that favours URLLC at the expense of lower-priority slices. M5-PC distributes resources more equitably, achieving mMTC satisfaction of 0.770 while maintaining URLLC at 0.870. M5-NoNeg shows intermediate performance but with substantially higher variance, indicating that the coordinator's one-shot allocation is less predictable than the structured negotiation outcomes.

4.3.4 Performance Overview

Figure 9 shows the remaining performance metrics for the three variants.

Token consumption is comparable across all three variants (approximately 125,000 per episode), confirming that the negotiation mechanism does not substantially increase the computational cost relative to the no-negotiation baseline. Decision latencies are also similar (6.5–8.1 seconds),

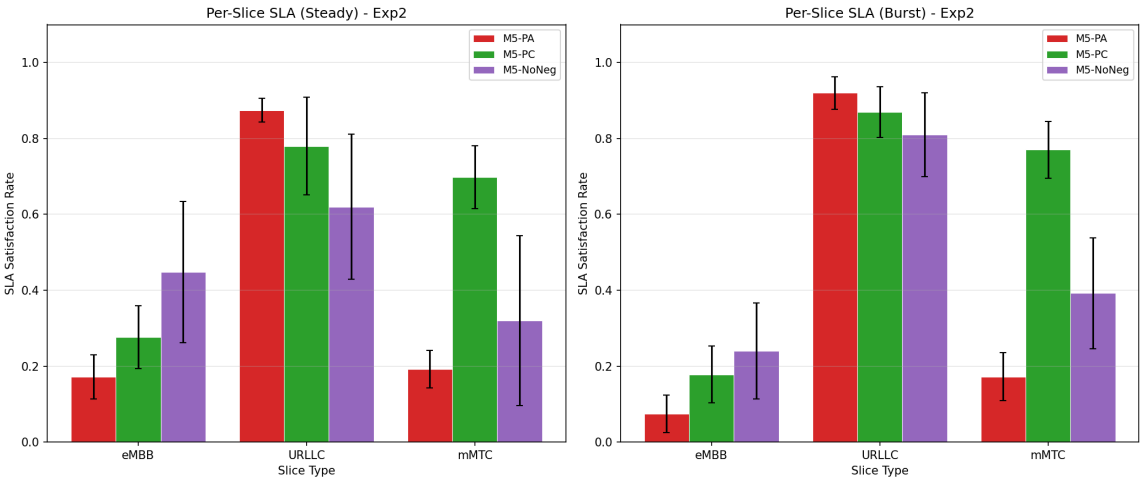


Figure 8: Experiment 2: Per-slice SLA satisfaction rates for multi-agent variants.

as the dominant cost is the LLM API calls for the three slice agents and the coordinator, which are present in all variants.

4.4 Result Analysis

4.4.1 Addressing RQ1: Multi-Agent LLM vs. Baselines

The experimental results present a nuanced picture regarding RQ1. Contrary to the initial hypothesis, the multi-agent LLM framework does not uniformly outperform all baselines across all metrics. The specific findings are:

1. **SLA satisfaction:** M5-PC achieves the second-highest average SLA satisfaction (0.606 burst, 0.585 steady), competitive with M1-Fixed (0.612 burst, 0.617 steady) and outperforming M3-PPO (0.515 burst, 0.499 steady) and M4 (0.481 burst, 0.500 steady). However, M5-PA underperforms most baselines due to its aggressive prioritisation of URLLC.

2. **Throughput:** LLM-based methods generally achieve higher total throughput than rule-based methods, with M5-PA reaching the highest values (5549 Mbps). This can be attributed to more aggressive allocation decisions that concentrate resources on high-capacity slices.

3. **Fairness:** Rule-based methods (M2-Threshold in particular, with fairness 0.923–0.956) generally achieve higher inter-slice fairness than LLM-based methods. Among the LLM methods, M5-PC achieves the best fairness (0.794–0.865), approaching but not exceeding the rule-based baselines.

4. **Operational cost:** LLM-based methods incur substantial decision latencies (5.8–8.9 seconds) and token costs (26,000–125,000 per episode), which represent significant practical

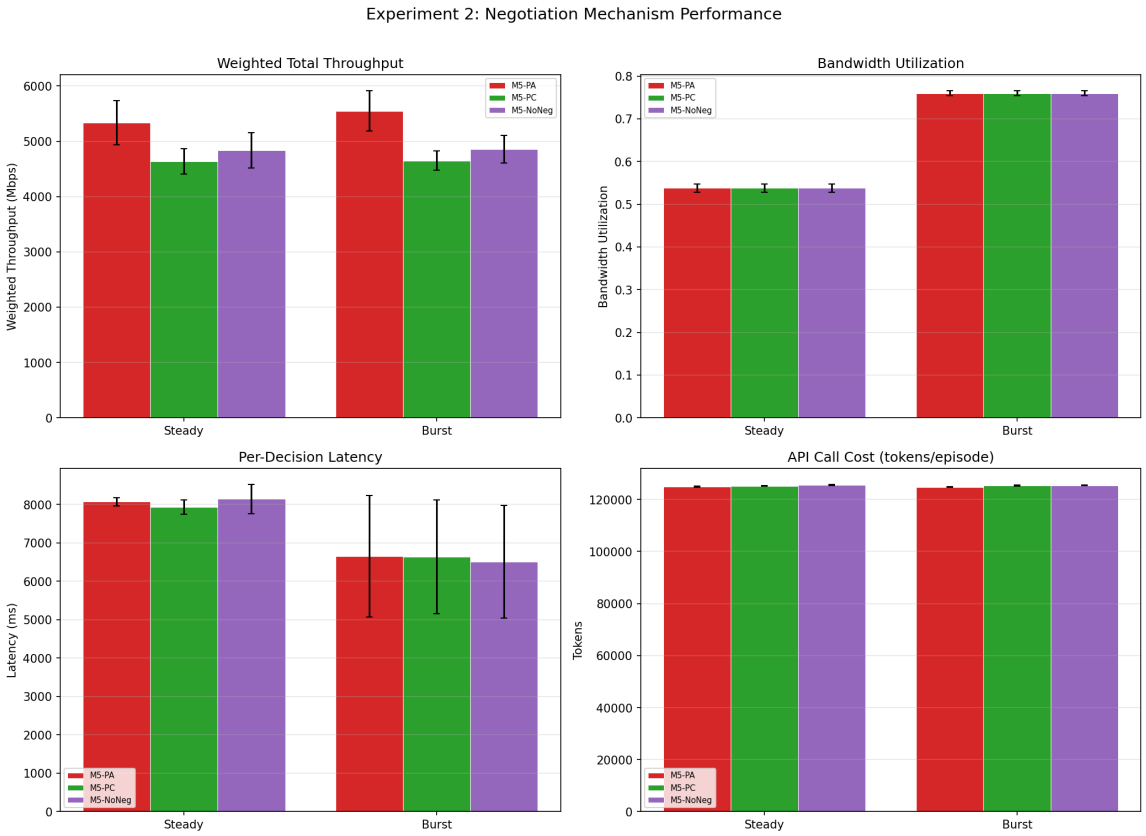


Figure 9: Experiment 2: Performance overview for multi-agent variants.

constraints that rule-based and DRL methods do not share.

These findings suggest that the choice of negotiation strategy is critical: M5-PC, with its proportional compromise approach, achieves competitive SLA satisfaction and reasonable fairness, while M5-PA's strict priority arbitration creates significant imbalances. The advantage of the multi-agent framework lies not in uniformly superior performance, but in its ability to reason about trade-offs between competing slice objectives in a more interpretable and configurable manner than monolithic approaches.

4.4.2 Addressing RQ2: Value of the Negotiation Mechanism

The comparison of M5-PC, M5-PA and M5-NoNeg provides evidence for the structural contribution of the negotiation mechanism:

1. M5-PC outperforms M5-NoNeg in average SLA satisfaction under both scenarios (0.606 vs. 0.481 under burst; 0.585 vs. 0.463 under steady), representing a relative improvement of approximately 26%.

2. M5-NoNeg exhibits substantially higher variance in fairness (± 0.104 under burst) com-

pared to M5-PC (± 0.055), indicating that the negotiation mechanism produces more consistent outcomes.

3. The token consumption and latency of M5-NoNeg are comparable to those of M5-PC, confirming that the performance differences are attributable to the negotiation protocol rather than to additional computation.

4. The choice between PA and PC negotiation strategies has a larger effect on performance than the presence or absence of negotiation itself, highlighting the importance of protocol design.

The negotiation mechanism, when combined with the Proportional Compromise strategy, provides measurable benefits in both SLA satisfaction and outcome consistency. The Priority Arbitration strategy, while effective for URLLC protection, does not yield balanced improvements across slices.

4.5 Threats to Validity and Limitations

Several factors may affect the generalisability of the reported results:

- **Simulation fidelity:** The environment uses simplified models for channel quality, throughput and SLA evaluation. Real-world network conditions involve more complex propagation effects, multi-cell interference and non-stationary traffic patterns that are not captured by the Poisson arrival model used here.

- **Scale of evaluation:** Each condition is evaluated with 5 random seeds, which limits the statistical power of comparisons. While the observed trends are consistent across seeds, larger-scale experiments would strengthen confidence in the findings.

- **LLM sensitivity:** The results depend on the specific LLM used (GPT-4 series) and may differ with other models. The temperature-0 setting reduces but does not eliminate stochastic variability in LLM outputs, and the prompt templates have not been optimised through systematic ablation.

- **Decision latency:** The per-step latencies of 5.8–8.9 seconds observed for LLM-based methods substantially exceed the 2-second threshold assumed in the two-tier design. In a real deployment, this would require either faster models, parallel API calls, or a larger decision interval.

- **Token cost:** The multi-agent framework consumes approximately 125,000 tokens per episode, which, at current API pricing, represents a non-trivial operational cost that scales with the number of decision cycles.

4.6 Practical Implications

The experimental results suggest several practical considerations for deploying LLM-based resource management in real networks:

- The Proportional Compromise negotiation strategy (M5-PC) achieves SLA satisfaction and fairness levels competitive with established baselines while offering superior interpretability through natural-language justifications and structured proposal exchanges.
- The choice of negotiation strategy has a larger impact on performance than the number of LLM agents or the number of negotiation rounds, suggesting that protocol design should be prioritised in future work.
- The substantial decision latencies observed (6–9 seconds per step) indicate that current LLM API infrastructure is not yet suitable for real-time network control at the timescales assumed in this work. Local or edge-deployed models, quantised inference, or hybrid LLM-RL approaches may be needed to bridge this gap.
- The interpretability advantage of LLM-based methods — where each agent produces a natural-language justification for its proposal — may be particularly valuable in regulated environments where allocation decisions must be auditable.

Chapter 5: Conclusion and Further Work

5.1 Conclusion

This project set out to design, implement and evaluate an LLM multi-agent negotiation framework for 5G network slice resource management, guided by two research questions. The conclusions drawn from the experimental results are summarised below.

RQ1: Can a multi-agent LLM framework match or exceed the performance of traditional rule-based strategies and DRL methods? The results show that the multi-agent framework with Proportional Compromise negotiation (M5-PC) achieves average SLA satisfaction rates competitive with the best-performing baselines (0.606 under burst, compared to 0.612 for M1-Fixed), while outperforming the PPO baseline (0.515) and the single-agent LLM (0.481) under burst conditions. However, no single LLM-based method uniformly dominates all baselines across all metrics. The M5-PA variant, using Priority Arbitration, achieves the highest throughput but at the cost of substantial fairness degradation. These findings indicate that the multi-agent LLM framework can be competitive with established methods, but its effectiveness is highly dependent on the choice of negotiation strategy.

RQ2: Does the structured negotiation mechanism provide measurable advantages? The comparison between M5-PC and M5-NoNeg demonstrates that the Proportional Compromise negotiation protocol improves average SLA satisfaction by approximately 26% under burst conditions (0.606 vs. 0.481) and reduces outcome variance in fairness by roughly 50% ($\pm$0.055 vs. $\pm$0.104). These improvements are achieved without additional token consumption, confirming that the negotiation protocol provides structural benefits beyond simply performing more LLM computations. The Protocol design matters considerably: Priority Arbitration (M5-PA) reduces overall fairness relative to the no-negotiation variant, illustrating that not all negotiation strategies are beneficial.

Beyond the direct answers to the research questions, this project has delivered three tangible outputs: (1) a reproducible Gymnasium-based simulation environment for 5G network slicing with clearly defined state space, action space and SLA evaluation logic; (2) a two-tier scheduling framework that decouples LLM reasoning from real-time execution; and (3) a systematic empirical comparison of six resource allocation methods across two traffic scenarios, providing a baseline for future work in this area.

5.2 Reflection

This project has been a substantial learning experience across several dimensions. From a technical perspective, implementing the multi-agent LLM framework required integrating skills in reinforcement learning (for the PPO baseline and environment design), prompt engineering (for

the agent system prompts), and distributed system design (for the LangGraph orchestration). Working with commercial LLM APIs introduced practical challenges not typically encountered in traditional software engineering, including non-deterministic outputs, rate limiting and the need for robust fallback mechanisms.

The experimental results provided an important lesson in intellectual honesty. The initial hypothesis that the multi-agent framework would uniformly outperform all baselines was not supported by the data. Rather than attempting to frame the results in a more favourable light, the analysis was designed to present the findings objectively, acknowledging both the strengths and limitations of the proposed approach. This experience reinforced the importance of pre-registering hypotheses and contingency analyses, as outlined in the mid-term report.

From a broader ethical perspective, the project raised considerations about the environmental cost of LLM inference. The multi-agent framework consumes approximately 125,000 tokens per episode, which, over multiple experimental runs, represents a non-trivial energy expenditure. This is a relevant concern for any LLM-based system intended for continuous network operation, and it motivated the inclusion of token consumption as an explicit evaluation metric.

The project also highlighted the tension between interpretability and efficiency. While the LLM-based methods produce natural-language justifications for their decisions — a clear advantage for auditability — the decision latencies observed (6–9 seconds per step) are incompatible with real-time network control under current infrastructure. This trade-off between interpretability and operational viability is likely to remain a central challenge as LLM-based network management matures.

5.3 Further Work

Several directions for future work emerge from the limitations and findings of this project:

- **Larger-scale and more realistic scenarios:** The current evaluation is limited to three slice types and 100 users. Extending to larger user populations, more slice types (e.g., vehicle-to-everything or augmented reality slices), and more complex traffic models (e.g., correlated arrivals, diurnal patterns) would test the scalability of the framework.
- **Adaptive negotiation strategies:** The current implementation uses fixed negotiation strategies (PA or PC). A natural extension would be to allow the coordinator to select or blend strategies based on the current network state, potentially learning an optimal strategy selection policy through reinforcement learning or meta-learning.
- **Edge-deployed or quantised models:** The decision latencies observed with cloud-based API calls suggest that deploying smaller, locally-hosted models could reduce latency to levels compatible with the two-tier design's assumptions. Evaluating the performance

trade-off between model size and allocation quality would be informative.

- **Online learning and continual adaptation:** Incorporating feedback from historical allocation outcomes into the agent prompts (e.g., through RAG or in-context learning) could enable the framework to adapt to long-term traffic trends without retraining.
- **Real-network or high-fidelity simulation validation:** Deploying the framework on an O-RAN testbed or a high-fidelity simulator such as ns-3 would provide more realistic validation and expose challenges related to multi-cell coordination, handover and physical-layer effects.
- **Hybrid LLM-RL approaches:** Combining the interpretability and reasoning capabilities of LLMs with the fast decision-making of trained RL policies — for example, using the LLM framework to set high-level allocation targets that an RL agent refines at shorter timescales — could offer the benefits of both paradigms.

References

[1] Afolabi I, Taleb T, Samdanis K, Ksentini A, Flinck H. Network Slicing and Softwarization: A Survey on Principles, Enabling Technologies, and Solutions. IEEE Communications Surveys & Tutorials. 2018;20(3):2429-53.

[2] Li R, et al. Deep Reinforcement Learning for Resource Management in Network Slicing. IEEE Access. 2018;6:74429-41.

[3] Sciancalepore V, Costa-Perez X, Banchs A. RL-NSB: Reinforcement Learning-Based 5G Network Slice Broker. IEEE/ACM Transactions on Networking. 2019;27(4):1543-57.

[4] Zhou H, et al. Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities. IEEE Communications Surveys & Tutorials. 2024.

[5] Tong J, et al.. WirelessAgent: Large Language Model Agents for Intelligent Wireless Networks; 2025. ArXiv:2505.01074. https://arxiv.org/abs/2505.01074.

[6] Shao J, et al. WirelessLLM: Empowering Large Language Models Towards Wireless Intelligence. Journal of Communications and Information Networks. 2024;9(2):99-112.

[7] Jiang F, et al. Large Language Model Enhanced Multi-Agent Systems for 6G Communications. IEEE Wireless Communications. 2024;31(6):48-55.

[8] Hong S, et al.. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework; 2023. ArXiv:2308.00352. https://arxiv.org/abs/2308.00352.

[9] Qian C, et al.. ChatDev: Communicative Agents for Software Development; 2023. ArXiv:2307.07924. https://arxiv.org/abs/2307.07924.

[10] Caballero P, Banchs A, Veciana GD, Costa-Perez X. Network Slicing Games: Enabling Customization in Multi-Tenant Mobile Networks. IEEE/ACM Transactions on Networking. 2019;27(2):662-75.

[11] Brockman G, et al.. OpenAI Gym; 2016. ArXiv:1606.01540. https://arxiv.org/abs/1606.01540.

[12] Nasser AR, Alani OY. Investigation of Multiple Hybrid Deep Learning Models for Accurate and Optimized Network Slicing. Computers. 2025;14(5):174.

[13] Lotfi F, Rajoli H, Afghah F. ORAN-GUIDE: RAG-Driven Prompt Learning for LLM-Augmented Reinforcement Learning in O-RAN Network Slicing; 2025. ArXiv:2506.00576. https://arxiv.org/abs/2506.00576.

[14] Qu Z, Wang W, Yu Z, Sun B, Li Y, Zhang X. LLM Enabled Multi-Agent System for 6G Networks: Framework and Method of Dual-Loop Edge-Terminal Collaboration; 2025. ArXiv:2509.04993. https://arxiv.org/abs/2509.04993.

[15] Lewis P, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS); 2020. .

[16] Wu D, et al. NetLLM: Adapting Large Language Models for Networking. In: Proceedings of the ACM SIGCOMM 2024 Conference; 2024. .

[17] Chen Y, et al. NetGPT: An AI-Native Network Architecture for Provisioning Beyond Personalized Generative Services. IEEE Network. 2024.

[18] Chergui H, Cid MC, Sayyad KP, Mur DC, Verikoukis C. Toward an Unbiased Collective Memory for Efficient LLM-Based Agentic 6G Cross-Domain Management; 2025. ArXiv:2509.26200. https://arxiv.org/abs/2509.26200.

[19] Wu Q, et al.. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation; 2023. ArXiv:2308.08155. https://arxiv.org/abs/2308.08155.

[20] Li G, Kader A, Itani H, Khizbullin D, Ghanem B. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society; 2023. ArXiv:2303.17760. https://arxiv.org/abs/2303.17760.

[21] Wang L, et al.. A Survey on Large Language Model based Autonomous Agents; 2023. ArXiv:2308.11432. https://arxiv.org/abs/2308.11432.

[22] Wei J, et al.. Chain of Thought Prompting Elicits Reasoning in Large Language Models; 2022. ArXiv:2201.11903. https://arxiv.org/abs/2201.11903.

[23] LangChain, Inc . LangGraph: Build Resilient Language Agents as Graphs; 2024. Accessed: Mar. 2026. https://github.com/langchain-ai/langgraph.

[24] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal Policy Optimization Algorithms; 2017. ArXiv:1707.06347. https://arxiv.org/abs/1707.06347.

[25] Raffin A, Hill A, Gleave A, Kanervisto A, Ernestus M, Dormann N. Stable-Baselines3: Reliable Reinforcement Learning Implementations; 2021. JMLR 22(268), 1–8. https://jmlr.org/papers/v22/20-1364.html.

[26] Jain R, Chiu D, Hawe W. A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. DEC Research Report TR-301. 1984.

Acknowledgment

I would like to express my sincere gratitude to my project supervisor for the invaluable guidance and support throughout the course of this project. From the initial brainstorming of research ideas to the refinement of the system design, my supervisor provided insightful direction that shaped the overall framework and methodology of this work. The constructive feedback on my writing, particularly regarding the structure and academic rigour of this report, was instrumental in improving its quality. I am also deeply grateful for the provision of GPU computing resources, which were essential for training the deep reinforcement learning baselines and running the large-scale multi-agent experiments that form the core of this study. Without this generous support, the experimental evaluation would not have been feasible within the project timeline.

I would also like to thank my classmates and friends who offered help and encouragement along the way. Their willingness to discuss technical problems, share practical experience with tools and frameworks, and provide moral support during challenging phases of the project contributed greatly to keeping the work on track.

Finally, I wish to acknowledge the open-source communities behind the libraries and platforms used in this project, including LangGraph, LangChain, Gymnasium, and Stable-Baselines3. Their well-maintained software and documentation made it possible to implement and evaluate the proposed framework efficiently.

Appendices

Disclaimer

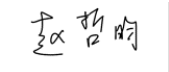
This report is submitted as part requirement for the undergraduate degree programme at Queen Mary University of London, and Beijing University of Posts and Telecommunications. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

BUPT No.: 2022213670

QMUL No.: 221170559

Full Name (Pin Yin): Zhao Zheyun

Full Name (Chinese): 赵哲昀

Signature: 

Date: 13-04-2026

Acknowledgement of using Generative AI

I acknowledge the use of the following Generative AI tools in the preparation of this report.

Publisher	**Name, version, URL**	**Use**	**Section**
Anthropic	Claude (Opus 4, Sonnet 4) `https://www.anthropic.com/claude`	To brainstorm the structure of the report, to assist in paraphrasing and proofreading the English writing, and to provide suggestions during the implementation of the simulation environment and multi-agent framework. All AI-assisted text was reviewed, edited and verified by the author.	Chapters 1–5; source code in `src/`
OpenAI	ChatGPT (GPT-5.2) `https://chatgpt.com/`	To search and summarise related literature during the background study, to clarify technical concepts in network slicing and reinforcement learning, and to debug LaTeX formatting issues. All cited references were independently verified by the author against their original sources.	Chapter 2; reference list

I confirm that the submitted work is my own, that the use of the above tools has been limited to the purposes described, and that all AI-generated content has been critically reviewed and, where appropriate, revised by the author before inclusion in this report.

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 1 - Supervisor

论文题目 Project Title	Network Resource Management Based on Language Model Multi-Agent Systems		
题目分类 Scope	Research	Networks and Wireless	Software
主要内容 Project description	This project explores network resource management based on language model multi-agent systems, aiming to address the growing complexity and dynamic demands of modern communication networks. With the rapid expansion of cloud services, data-intensive applications, and heterogeneous infrastructures, efficient resource allocation has become a critical challenge. Traditional centralized management approaches often struggle with scalability, adaptability, and real-time responsiveness. To overcome these limitations, this project leverages large language models (LLMs) integrated with multi-agent systems to enable distributed, intelligent decision-making. Each agent, powered by an LLM, is capable of interpreting high-level requirements, reasoning about network conditions, and collaboratively negotiating resource usage. The methods involve task decomposition, semantic reasoning, and reinforcement learning-based optimization for adaptive scheduling and load balancing. Tools such as Python, simulation frameworks (e.g., ns-3), and machine learning libraries (e.g., PyTorch) will be employed. By combining natural language understanding, multi-agent coordination, and algorithmic optimization, the project aims to deliver a scalable, resource-efficient, and intelligent management framework for next-generation communication networks.		
关键词 Keywords	Network Resource Managemen, Language Model, Multi-Agent Systems		
主要任务 Main tasks	1 Requirement Analysis and Modeling Identify the key challenges in network resource management and model them as tasks suitable for language model multi-agent systems.		
	2 Design and Development of Multi-Agent Framework Build a framework where language model agents can interpret network requirements, interact, and coordinate resource allocation decisions.		
	3 Implementation of Algorithms and Tools Apply optimization techniques, reinforcement learning, and semantic reasoning within the multi-agent system using tools such as Python, PyTorch, and network simulators.		
	4 Evaluation and Performance Validation Test the framework in simulated network environments (e.g., ns-3) to measure scalability, efficiency, and adaptability, and compare results against traditional approaches.		
主要成果 Measurable outcomes	1 Practical Skills in AI and Networking Students will gain hands-on experience with large language models, multi-agent system design, and network simulation tools, building interdisciplinary expertise in AI-driven network management.		
	2 Ability to Design and Evaluate Intelligent Systems Students will learn how to design algorithms for resource allocation, implement optimization and		

	reinforcement learning methods, and critically evaluate system performance in complex environments.
	3 The multi-agent demo system for resouce allocation simulation.

北京邮电大学 本科毕业设计（论文）任务书

Project Specification Form

Part 2 - Student

学院 School	International School	**专业 Programme**	**Intelligence Science and Technology**		
姓 Family name	赵 Zhao	**名 First Name**	哲昀 Zheyun		
BUPT 学号 BUPT number	2022213670	**QM 学号 QM number**	221170559	**班级 Class**	2022215124
论文题目 Project Title	Network Resource Management Based on Language Model Multi-Agent Systems				
论文概述 Project outline **Write about 500-800 words** **Please refer to Project Student Handbook section 3.2**	**Background and Motivation** This project focuses on accelerating the communication pathway of KVCache transmission in large language model (LLM) training and inference, using the Huawei Ascend 910C NPU as the primary hardware platform [1]. In modern LLM systems—especially those deployed with high-performance distributed training or inference frameworks—the transfer of KVCache (Key–Value Cache) across devices has become a major performance bottleneck [2]. The challenge becomes more significant under the Prefill-Decode Separation (PD Separation) architecture. PD Separation decouples the model execution stages into Prefill (processing the input prompt and generating KVCache) and Decode (autoregressive token generation using the cached keys and values) [3]. This architecture improves parallelism and resource utilization but also increases the volume and frequency of KVCache transfers between system components [4]. **Limitations of Existing Communication Mechanisms** Traditional communication mechanisms such as RDMA (Remote Direct Memory Access) often struggle to meet the bandwidth and latency demands of KVCache transmission in PD-based systems. These limitations include insufficient effective bandwidth, fluctuations in latency, suboptimal path utilization, and scaling constraints when multiple workers are involved [5]. To overcome these challenges, this project aims to leverage SIO (Serial Input/Output)—a fundamental serial communication interface consisting of a one-to-one link between a master device and a slave device via a single data line and a clock line. The internal SIO interconnect of Ascend 910C offers high bandwidth and low latency, making it a promising alternative for high-speed KVCache transmission. **Project Objective** The key objective of the project is to design, implement, and integrate a new SIO-based communication module into the vLLM framework to accelerate KVCache transfer [2]. The target performance metric is to achieve at least 4× effective bandwidth improvement compared with the RDMA-based baseline in representative PD-separated scenarios. **Requirement Analysis and Bottleneck Identification** The project begins with an initial analysis of user/system requirements and identification of KVCache transmission bottlenecks. This stage includes profiling KVCache size patterns, understanding vLLM's KV reuse and dispatching mechanisms, and evaluating the limitations of the RDMA communication path.				

	Data will be collected through Ascend 910C profiling tools, custom communication benchmarks, and vLLM execution traces [6]. **Algorithms and Methodologies** Next, various algorithms and methodologies will be explored. These include: • Analyzing SIO hardware characteristics (serial master–slave structure, clock-driven data synchronization, pipeline feasibility). • Designing optimized transmission strategies such as pipelined KV slicing, asynchronous forwarding, multilayer parallel scheduling, and adaptive buffer allocation [7]. • Building a plug-and-play communication abstraction that allows easy switching between RDMA mode and SIO mode within vLLM [8]. • Integrating the SIO communication backend into vLLM's KVCache sharing modules (e.g., distributed KV reuse, remote KV transfer APIs) [9]. **System Integration and User Interaction** In terms of system interaction, users will be able to enable the SIO-accelerated transmission path via configuration or through an API interface in vLLM. The system should automatically detect hardware topology, initialize SIO channels, and manage transmission logic transparently. **Experimental Design and Evaluation** The experimental component will involve developing a reproducible benchmark suite capable of evaluating KVCache transmission under different conditions, such as varying KV dimensions, sequence lengths, numbers of layers, and numbers of decoding workers. Metrics including raw bandwidth, end-to-end latency, throughput, prefill-to-decode delay, and NPU/CPU utilization will be measured. These results will be compared directly against the RDMA-based baseline to verify whether the target speedup is achieved [10]. **Tools, Languages, and Platform** The project will use C++ and Python as the primary programming languages. Key tools include the Ascend CANN toolkit, AscendCL APIs, communication profiling utilities, and the vLLM execution engine. Background reference materials will include PD Separation literature, vLLM documentation, Ascend 910C communication manuals, and prior work on KVCache optimization and high-performance interconnects. **References** 1. Zuo P, Lin H, Deng J, et al. Serving Large Language Models on Huawei CloudMatrix384. arXiv preprint [Internet]. 2025 [cited 2025 Nov 17]; arXiv:2506.12708. Available from: https://arxiv.org/abs/2506.12708 2. Kwon W, Li Z, Zhuang S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM). arXiv preprint [Internet]. 2023 [cited 2025 Nov 17]; arXiv:2309.06180. Available from: https://arxiv.org/abs/2309.06180 3. Zhong Y, Liu S, Chen J, et al. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. arXiv preprint [Internet]. 2024 [cited 2025 Nov 17]; arXiv:2401.09670. Available from: https://arxiv.org/abs/2401.09670 4. Xiong Y, Wu H, Shao C, et al. LayerKV: Optimizing Large Language Model Serving with Layer-wise KV Cache Management. arXiv preprint [Internet]. 2024 [cited 2025 Nov 17]; arXiv:2410.00428. Available from: https://arxiv.org/abs/2410.00428

<table>
<tr><td></td><td>5. Li H, Li Y, Tian A, et al. A Survey on Large Language Model Acceleration Based on KV Cache Management. arXiv preprint [Internet]. 2024 [cited 2025 Nov 17]; arXiv:2412.19442. Available from: https://arxiv.org/abs/2412.19442
6. Cheng Y, Yang J, Zhang R, Liu F. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM. arXiv preprint [Internet]. 2025 [cited 2025 Nov 17]; arXiv:2510.09665. Available from: https://arxiv.org/abs/2510.09665
7. Liao B, Zhang S, Wang Y, et al. Beyond KV Caching: Shared Attention for Efficient LLMs. arXiv preprint [Internet]. 2024 [cited 2025 Nov 17]; arXiv:2407.12866. Available from: https://arxiv.org/abs/2407.12866
8. Zhao J, Li J, Wu C. Sandwich: Separating Prefill-Decode Compilation for Efficient CPU LLM Serving. arXiv preprint [Internet]. 2025 [cited 2025 Nov 17]; arXiv:2507.18454. Available from: https://arxiv.org/abs/2507.18454
9. Yang J, Xu B, Li N, et al. KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse. arXiv preprint [Internet]. 2025 [cited 2025 Nov 17]; arXiv:2502.16002. Available from: https://arxiv.org/abs/2502.16002
10. Du D, Cao S, Cheng J, Mai L, Cao T, Yang M. BitDecoding: Unlocking Tensor Cores for Long-Context LLMs with Low-Bit KV Cache. arXiv preprint [Internet]. 2025 [cited 2025 Nov 17]; arXiv:2503.18773. Available from: https://arxiv.org/abs/2503.18773</td></tr>
<tr><td rowspan="2">道德规范
Ethics

Please discuss ethical issues with your supervisor.

Please refer to Project Student Handbook section 4.1 and Project Ethics Guidance</td><td>Please confirm by checking the box:

☒ I confirm that I have discussed ethical issues with my supervisor.</td></tr>
<tr><td>Summary of ethical issues:

1. Data Privacy and Security
KVCache represents intermediate model states and may implicitly contain user-related semantic information. All experiments must use non-sensitive datasets, and secure data handling practices must be followed in accordance with university and Ascend platform requirements.
2. Algorithmic Transparency and Reproducibility
The SIO-accelerated communication mechanism MUST NOT alter model semantics or change vLLM’s inference outputs. All optimization logic, fallback mechanisms, and communication paths must be clearly documented and reproducible.
3. Fairness and Responsible Deployment
Communication-layer optimization must not introduce unfair resource allocation (e.g., prioritizing certain requests) or system instability. The design must consider safe fallback to RDMA mode in case of failure.
4. Maintainability and Auditability
The codebase should include proper documentation, tests, version tracking, and monitoring logs to enable debugging, auditing, and long-term maintenance.
5. Societal Impact and Sustainability
Improvements in communication performance may enable larger-scale LLM deployments. The project should consider energy efficiency, resource consumption, and responsible scaling practices.</td></tr>
</table>

中期目标 **Mid-term target.** **It must be tangible outcomes, E.g. software, hardware or simulation.** **It will be assessed at the mid-term oral.**	By the mid-point of the project, the following tangible deliverables are expected: **1. SIO Communication Prototype** A working SIO master-slave communication module on Ascend 910C. Successful basic data transfer demo validating SIO channel initialization and transmission. **2. KVCache Transmission Benchmark Suite** Tools to simulate KVCache of different sizes and shapes (token length, number of layers, head dimensions). Supports both RDMA and SIO modes with automated performance measurement (bandwidth, latency, throughput). **3. Preliminary System Profiling Report** Analysis of RDMA bottlenecks in PD Separation scenarios. Graphs showing bandwidth curves, prefill-to-decode pipeline timing, and resource usage. **4. Initial Thesis Draft** Completed outline including Background, Problem Definition, Technical Approach, and Experiment Design.

Work Plan (Gantt Chart)

Fill in the sub-tasks and insert a letter X in the cells to show the extent of each task

	Nov 1-15	Nov 16-30	Dec 1-15	Dec 16-31	Jan 1-15	Jan 16-31	Feb 1-15	Feb 16-28	Mar 1-15	Mar 16-31	Apr 1-15	Apr 16-30
Task 1 Requirement Analysis and Modeling Identify the key challenges in network resource management and model them as tasks suitable for language model multi-agent systems.												
Analyse PD-Separation (Prefill–Decode) workflow and KVCache transmission patterns	X	X										
Benchmark RDMA-based KVCache transmission & identify bottlenecks	X	X	X									
Study Ascend 910C SIO hardware characteristics and communication constraints		X	X	X								
Review vLLM's KVCache reuse/dispatching architecture			X	X								
Task 2 Design and Development of Multi-Agent Framework Build a framework where language model agents can interpret network requirements, interact, and coordinate resource allocation decisions.												
Design SIO-based KVCache transmission architecture					X	X						
Define data slicing / pipelining mechanism for high-throughput KV transfer						X	X					
Develop communication abstraction layer (switchable RDMA ↔ SIO)							X	X				
Integrate SIO backend with vLLM's KVCache transport interface								X	X			
Task 3 Implementation of Algorithms and Tools Apply optimization techniques, reinforcement learning, and semantic reasoning within the multi-agent system using tools such as Python, PyTorch, and network simulators.												
Code SIO master-slave communication module								X	X			
Implement KVCache sender / receiver pipelines									X	X		
Develop profiling suite (BW / latency)										X	X	
Optimize buffers / parallelism / async scheduling											X	X
Task 4 Evaluation and Performance Validation Test the framework in simulated network environments (e.g., ns-3) to measure scalability, efficiency, and adaptability, and compare results against traditional approaches.												
Prepare experimental bed (Ascend 910C + vLLM)										X	X	

Run PD pipeline latency tests											X	X
Compare RDMA vs SIO throughput & Bandwidth												X
Analyse results & write performance report												X

北京邮电大学 本科毕业设计（论文）初期进度报告

Project Early-term Progress Report

学院 School	International School	**专业 Programme**	**Intelligent Science and Technology**		
姓 Family name	赵 Zhao	**名 First Name**	哲昀 Zheyun		
BUPT 学号 BUPT number	2022213670	**QM 学号 QM number**	221170559	**班级 Class**	2022215124
论文题目 Project Title	Network Resource Management Based on Language Model Multi-Agent Systems				

Literature Review

1. Background and Motivation

Large Language Model (LLM)-powered applications are rapidly evolving from single-request inference services into complex **multi-agent systems**, where multiple specialized agents collaborate through structured workflows to solve sophisticated tasks such as question answering, planning, code generation, and retrieval-augmented reasoning. In such systems, a single user request may trigger a sequence of LLM invocations, tool calls, and inter-agent communications, making **end-to-end (E2E) response latency** the primary determinant of user experience.

At the same time, modern LLM services are increasingly deployed in **shared, multi-tenant public cloud environments**, where multiple applications and agents contend for the same underlying inference infrastructure. Under high or bursty loads, resource contention at both the compute level (e.g., GPU queues, memory pressure) and the communication level (e.g., network latency, cross-node traffic) can significantly amplify queueing delays and tail latency. These challenges motivate a growing body of research on **end-to-end optimization of LLM-based applications**, shifting focus from isolated inference efficiency toward system-level coordination across workflows, resources, and execution environments.

This literature review surveys recent representative work in this area, focusing on five closely related research directions: (1) workflow-aware multi-agent serving under shared resources, (2) application semantic exposure for end-to-end optimization, (3) inference-stage disaggregation for improved service goodput, (4) fine-grained workflow orchestration frameworks, and (5) online network latency estimation using exponentially weighted moving averages (EWMA). By analyzing the strengths and limitations of existing approaches, this review identifies a clear research gap at the intersection of **multi-agent workflow scheduling and dynamic network-aware resource management**, which motivates further investigation.

2. Workflow-Aware Multi-Agent Serving under Shared LLM Resources

As multi-agent applications increasingly rely on shared LLM backends, a key challenge lies in coordinating heterogeneous agent workloads under resource contention. Kairos directly addresses this problem by targeting **low-latency multi-agent serving with shared LLMs under excessive loads** [1]. The central observation of Kairos is that different agents within the same application exhibit substantially different inference characteristics, including prompt lengths, output lengths, latency distributions, and GPU memory demands. However, existing serving systems often schedule requests in a first-come-first-served (FCFS) manner and distribute them across instances without considering such inter-agent differences, leading to inefficient queueing and increased tail latency.

To address this issue, Kairos introduces a workflow-aware serving architecture composed of three main components: an online workflow orchestrator that reconstructs dynamic agent workflows at runtime, a priority scheduler that orders requests based on agent-specific remaining latency distributions, and a memory-aware dispatcher that assigns requests to LLM instances while avoiding GPU memory overcommitment [1]. By explicitly incorporating workflow structure and

agent heterogeneity into the scheduling loop, Kairos demonstrates significant reductions in end-to-end latency compared to prior end-to-end optimization systems.

Despite these contributions, Kairos primarily focuses on **compute-side bottlenecks**, particularly GPU queueing and memory pressure. Its dispatching decisions are largely driven by estimated memory feasibility and expected execution time, while network conditions between agents and LLM instances are not explicitly modeled. In distributed or multi-zone deployments, network latency can become a dominant factor in end-to-end performance, suggesting an opportunity to extend workflow-aware serving frameworks with network-aware resource management.

3. Exposing Application Semantics for End-to-End Optimization

A fundamental obstacle to end-to-end optimization is the limited visibility that serving systems have into application-level structure. Parrot addresses this limitation by arguing that traditional request-based LLM APIs conceal crucial semantic relationships between calls, preventing system-level optimizations across an application's execution graph [2]. To overcome this, Parrot introduces the concept of **semantic variables**, which explicitly encode the data dependencies between LLM invocations within an application.

By exposing these semantic variables to the serving layer, Parrot enables the system to identify redundant computations, exploit data reuse, and apply pipeline and batching optimizations across related requests. The system demonstrates that making application semantics explicit can significantly improve throughput and latency without requiring changes to the underlying LLM models [2].

While Parrot successfully expands the optimization space by bridging the application–system boundary, its primary emphasis lies in **semantic dataflow exploitation** rather than resource contention management under high load. In particular, Parrot does not explicitly address scenarios where multiple applications or agents compete for shared LLM infrastructure, nor does it consider dynamic network conditions that may influence end-to-end latency in distributed deployments.

4. Fine-Grained Workflow Orchestration for LLM Applications

Complementary to Parrot's semantic exposure approach, Teola focuses on the **granularity of orchestration** in LLM-based applications [3]. The authors argue that treating entire application modules as indivisible units limits optimization opportunities, especially when workflows contain both LLM and non-LLM components. Teola proposes decomposing applications into **primitive-level dataflow graphs**, enabling more aggressive parallelization and pipeline execution across workflow stages.

Building on this idea, Ayo serves as a system prototype that implements fine-grained orchestration for end-to-end LLM applications, demonstrating practical performance gains in real workloads [4]. These systems highlight the importance of capturing workflow structure at a sufficiently fine granularity to enable meaningful end-to-end optimization.

However, similar to Parrot, Teola and Ayo primarily address **execution structure and scheduling granularity**, with limited attention to resource heterogeneity and dynamic network effects. As LLM applications scale across distributed clusters or geographically separated regions, network latency and bandwidth variability may undermine the benefits of fine-grained orchestration unless explicitly incorporated into the scheduling model.

5. Disaggregating Inference Stages for Goodput-Oriented Serving

In parallel with workflow-level optimizations, significant effort has been devoted to improving the efficiency of LLM inference services themselves. DistServe proposes a novel serving architecture that **disaggregates the prefill and decoding stages** of LLM inference, allowing each stage to be optimized independently under different latency constraints [5]. The motivation behind DistServe is that co-locating prefill and decoding forces a compromise between time-to-first-token (TTFT) and time-per-output-token (TPOT), limiting achievable goodput.

By separating these stages and placing them on different GPU resources, DistServe enables more flexible resource allocation and better alignment with application-level service-level objectives (SLOs). Importantly, DistServe explicitly considers **communication overhead** between

disaggregated stages and leverages network bandwidth characteristics when making placement decisions [5].
DistServe demonstrates that internal inference architecture plays a critical role in end-to-end performance. Nevertheless, its scope is largely confined to the inference service layer. The system does not directly address higher-level multi-agent workflows, nor does it consider how dynamic network conditions might interact with workflow-level scheduling and routing decisions.

6. EWMA-Based Network Latency Estimation and Change Detection

Effective network-aware resource management requires reliable online estimation of network latency. Exponentially Weighted Moving Average (EWMA) techniques provide a well-established foundation for such estimation. Early work by Roberts introduced exponentially weighted averages as a method for detecting small shifts in process behavior [6], while subsequent studies by Lucas and Saccucci analyzed the statistical properties and robustness of EWMA control schemes [7].
In networking systems, EWMA has been widely adopted for smoothing round-trip time (RTT) measurements. Jacobson and Karels applied exponential smoothing to RTT estimation as part of TCP congestion control mechanisms, enabling stable retransmission timeout calculation despite noisy measurements [8]. This approach was later standardized in TCP retransmission timer computation specifications [9].
The appeal of EWMA lies in its simplicity, low computational overhead, and ability to distinguish persistent trends from transient fluctuations. These properties make EWMA particularly suitable for **online network monitoring in distributed systems**, where rapid yet stable responses to changing conditions are required. However, EWMA itself is a building block rather than a complete solution; its effectiveness depends on how smoothed estimates are integrated into higher-level scheduling and routing policies.

7. Summary and Identified Research Gap

Collectively, the reviewed literature demonstrates substantial progress toward end-to-end optimization of LLM-based applications. Workflow-aware serving systems such as Kairos highlight the importance of agent heterogeneity and shared-resource management under load [1]. Parrot and Teola/Ayo emphasize the necessity of exposing application structure and increasing orchestration granularity to unlock end-to-end performance gains [2–4]. DistServe illustrates how inference-stage disaggregation and SLO-aware resource allocation can improve service efficiency [5]. Meanwhile, EWMA-based methods provide a robust foundation for online network latency estimation [6–9].
Despite these advances, a notable gap remains. Existing systems predominantly optimize **compute-side resources and execution structure**, while **network dynamics**—including topology changes, link degradation, and cross-node communication costs—are often treated as secondary considerations or assumed to be static. In distributed multi-agent settings, however, network conditions can significantly influence end-to-end latency, especially under shared infrastructure and dynamic workloads.
This gap motivates further research into **network-aware resource management for multi-agent LLM workflows**, integrating online network latency estimation with workflow-aware scheduling and resource dispatching. Addressing this challenge has the potential to complement existing approaches and provide more robust end-to-end latency guarantees in realistic, dynamic deployment environments.

References

1. Chen J, Shi J, Chen Q, Guo M. *Kairos: Low-latency Multi-Agent Serving with Shared LLMs and Excessive Loads in the Public Cloud*. arXiv preprint [Internet]. 2025 [cited 2026 Jan 11]; arXiv:2508.06948. Available from: https://arxiv.org/abs/2508.06948
2. Lin C, Han Z, Zhang C, Yang Y, Yang F, Chen C, et al. *Parrot: Efficient Serving of LLM-based Applications with Semantic Variable*. arXiv preprint [Internet]. 2024 [cited 2026 Jan 11]; arXiv:2405.19888. Available from: https://arxiv.org/abs/2405.19888
3. Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. arXiv preprint

[Internet]. 2024 [cited 2026 Jan 11]; arXiv:2401.09670. Available from: https://arxiv.org/abs/2401.09670
4. Tan X, Jiang Y, Yang Y, Xu H. *Teola: Towards End-to-End Optimization of LLM-based Applications*. arXiv preprint [Internet]. 2024 [cited 2026 Jan 11]; arXiv:2407.00326. Available from: https://arxiv.org/abs/2407.00326
5. Tan X, Jiang Y, Yang Y, Xu H. *Towards End-to-End Optimization of LLM-based Applications with Ayo*. Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25) [Internet]. 2025 [cited 2026 Jan 11];10.1145/3676641.3716278. Available from: https://doi.org/10.1145/3676641.3716278
6. Roberts SW. *Control Chart Tests Based on Geometric Moving Averages*. Technometrics [Internet]. 1959 [cited 2026 Jan 11];1(3):239–250. Available from: https://www.jstor.org/stable/1266443
7. Lucas JM, Saccucci MS. *Exponentially Weighted Moving Average Control Schemes: Properties and Enhancements*. Technometrics [Internet]. 1990 [cited 2026 Jan 11];32(1):1–12. Available from: https://www.stat.cmu.edu/technometrics/90-00/vol-32-01/v3201001.pdf
8. Jacobson V, Karels MJ. *Congestion Avoidance and Control*. In: Proceedings of the ACM SIGCOMM Conference [Internet]. 1988 [cited 2026 Jan 11]. Available from: https://ee.lbl.gov/papers/congavoid.pdf
9. Paxson V, Allman M, Chu J, Sargent M. *Computing TCP's Retransmission Timer*. RFC 6298 [Internet]. 2011 [cited 2026 Jan 11]. Available from: https://www.rfc-editor.org/rfc/rfc6298

Summary of the Progress of the Project So Far

To date, the project has focused on establishing a solid theoretical and conceptual foundation for subsequent system design and implementation. The primary progress can be summarized in the following aspects.

First, an extensive review of state-of-the-art literature related to multi-agent systems, large language model (LLM) serving, and end-to-end latency optimization has been conducted. Key representative works, including Kairos, Parrot, DistServe, and Teola/Ayo, have been carefully studied to understand current research directions, system architectures, and optimization strategies. Through this process, the project has identified how existing approaches address workflow-aware scheduling, resource sharing under high load, inference-stage optimization, and end-to-end performance considerations, as well as their limitations, particularly with respect to dynamic network conditions.

Second, the project has focused on acquiring and consolidating domain-specific terminology and foundational concepts relevant to this research area. This includes, but is not limited to, concepts such as multi-agent workflows, end-to-end latency, tail latency, service-level objectives (SLOs), inference pipeline stages (prefill and decoding), resource contention, and basic network performance metrics. A clear understanding of these terms has enabled more precise interpretation of prior work and has helped establish a consistent technical vocabulary for subsequent analysis and system design.

Overall, the work completed so far has clarified the research background, identified key challenges in existing systems, and laid the conceptual groundwork necessary for defining a concrete technical approach in the next phase of the project.

是否符合进度？On schedule as per GANTT chart?

YES

下一步 Next steps:

The next phase of the project will focus on transitioning from literature understanding to problem formalization and preliminary system design. The immediate planned tasks are as follows.

First, the project will refine and formalize the specific research problem to be addressed, with particular emphasis on end-to-end latency optimization in multi-agent LLM systems under dynamic network conditions. This includes clearly defining the system model, the scope of network variability considered, and the performance metrics used for evaluation.

Second, based on insights gained from the literature review, an initial system architecture will be designed. This architecture will outline the roles of core components such as workflow tracking, request dispatching, and network-aware decision-making, and will specify how network-related information can be incorporated into the resource management process.

Third, the project will begin preparing the experimental and implementation environment. This includes selecting appropriate tools or frameworks for prototyping, identifying candidate workloads or benchmark scenarios, and planning how network conditions can be monitored or emulated for evaluation purposes.

These steps will provide a concrete foundation for subsequent implementation and experimentation, which will be the focus of later stages of the project.

北京邮电大学 本科毕业设计（论文）中期进度报告

Project Mid-term Progress Report

学院 School	International School	**专业 Programme**	**Intelligent Science and Technology**		
姓 Family name	赵 Zhao	**名 First Name**	哲昀 Zheyun		
BUPT 学号 BUPT number	2022213670	**QM 学号 QM number**	221170559	**班级 Class**	2022215124
论文题目 Project Title	Network Resource Management Based on Language Model Multi-Agent Systems				

是否完成任务书中所定的中期目标？Targets met (as set in the Specification)?
YES

已完成工作 Finished work:

1. Research Survey and Problem Definition

5G network slicing technology enables the creation of multiple logical networks over a shared physical infrastructure, with each slice serving differentiated service requirements [13]. Enhanced Mobile Broadband (eMBB) pursues high throughput, Ultra-Reliable Low-Latency Communication (URLLC) is designed for extremely low-latency and high-reliability scenarios, typically characterised by millisecond-level end-to-end latency and stringent reliability targets, and massive Machine-Type Communication (mMTC) must support concurrent access from massive numbers of devices. When these three slice types share limited spectrum resources, their Quality of Service (QoS) requirements become inherently competitive. Heavy bandwidth consumption by eMBB users streaming high-definition video may directly threaten latency guarantees for URLLC users, while periodic mass reporting by mMTC devices can squeeze access resources available to other slices [13]. Traditional network resource management methods, including rule-based heuristic allocation and Deep Reinforcement Learning (DRL)-based optimisation, encounter significant challenges [14, 15]. Although such approaches can encode constraints and priorities explicitly, they generally struggle to integrate 3rd Generation Partnership Project (3GPP) Service Level Agreement (SLA) constraints, operator-defined business priority policies, and dynamically evolving user behaviour patterns into a unified and adaptive decision-making framework.

The emergent capabilities of Large Language Models (LLMs) offer a new approach to this problem [17]. Since 2023, researchers have begun exploring LLMs as intelligent decision engines for network management. However, existing work has revealed two core gaps that form the starting point of this research.

Research Gap 1: Lack of an end-to-end multi-agent LLM framework. The WirelessAgent framework, implements a complete cognitive architecture consisting of perception, memory, planning, and action, and demonstrates superior performance in network slicing scenarios, reporting bandwidth utilisation 44.4% higher than pure prompting methods under its experimental setting [1]. However, it remains fundamentally a single-agent design in which one LLM is responsible for resource decisions across all slices. Such a centralised structure lacks specialised analytical roles and structured negotiation mechanisms when inter-slice demand conflicts arise. CommLLM proposes a three-component multi-agent architecture, with validation primarily focused on semantic communication rather than network slice resource allocation [3]. Consequently, a complete framework dedicated to network slice resource management that supports explicit multi-agent collaboration and can be validated in an end-to-end slicing environment remains underexplored and warrants further investigation.

Research Gap 2: Lack of structured multi-agent negotiation and conflict resolution mechanisms. Network slice resource allocation can be formulated as a constrained multi-party optimisation problem in which total bandwidth is fixed and slices compete for limited resources [19]. This setting differs substantially from the predominantly cooperative task structures addressed by general-purpose multi-agent LLM frameworks such as MetaGPT [9] and ChatDev [12]. In competitive slicing scenarios, agents must reconcile conflicting objectives under hard resource constraints and service-level requirements. Systematic negotiation protocols and conflict resolution mechanisms tailored to such competitive environments remain underexplored in the network management literature.

2. Research Objective and Contributions

Research Objective: In a 5G network slicing simulation environment, design and implement an LLM multi-agent negotiation-based resource allocation framework (two-tier scheduling with low-frequency LLM multi-agent policy decisions + high-frequency rule execution), compare it against rule-based and DRL baselines, and validate the improvement of the multi-agent negotiation mechanism in SLA satisfaction rate and resource utilisation.

Core Contributions:

1. **A self-built reproducible 5G network slicing resource management simulation environment** (based on Gymnasium) [20], with clearly defined state space, action space, reward function, and SLA evaluation logic, with all parameters and interfaces made publicly available.
2. **A two-tier scheduling LLM multi-agent decision framework**: the upper tier features multiple specialised LLM agents (slice agents + coordinator agent) collaborating on resource allocation decisions through a negotiation protocol; the lower tier uses a rule executor operating at millisecond granularity. Framework effectiveness is validated through 3 core comparative experiments.

Research Questions:

RQ1: Can a multi-agent LLM framework outperform traditional rule-based strategies and DRL methods in network slice resource management? Corresponding objective: Quantify the performance differences of the multi-agent LLM framework relative to rule-based and DRL baselines in SLA satisfaction rate, resource utilisation, and dynamic load adaptability through comparative experiments.

RQ2: Does the negotiation mechanism between agents provide structural advantages? Corresponding objective: Through a "with negotiation vs. without negotiation (centralised dictatorial allocation)" comparison, validate the value contribution of the negotiation protocol itself, rather than merely the result of "more LLM calls providing more computation."

3. Literature Review

3.1 Diverging Technical Routes for LLM Agent Frameworks

Between 2023 and 2025, LLM agent research for wireless networks has developed along three technical routes, each making different trade-offs in addressing domain adaptation.

The prompting and agent workflow route is represented by WirelessAgent [1] and WirelessLLM [2]. WirelessAgent decomposes network management tasks into a sequential workflow of intent understanding → slice allocation → bandwidth allocation → load balancing, with each stage completed by a LangGraph node managing global state transitions. The core contribution is demonstrating that "cognitive architecture + tool calling" can approach rule-optimal performance without training (only 4.3% lower). WirelessLLM proposes a more complete methodological

roadmap: rapid domain knowledge injection through prompt engineering and RAG (knowledge alignment), fusion with specialised models (knowledge fusion), and model evolution through continual learning [2].

The parameter fine-tuning route is represented by NetLLM [4], which reformulates network optimisation as sequential decision problems. Based on Llama2-7B and LoRA, it outperforms DRL baselines on viewport prediction, adaptive bitrate selection, and cluster job scheduling. However, this route requires large amounts of labelled data, limiting its applicability in data-scarce network slicing scenarios.

The cloud-edge collaboration route was pioneered by NetGPT [5], proposing edge deployment of lightweight LLMs for latency-sensitive inference and cloud deployment of large models for global optimisation.

3.2 Multi-Agent Collaboration: From General Paradigms to Network Adaptation

General-purpose multi-agent LLM research has formed relatively mature methodologies. MetaGPT encodes Standard Operating Procedures (SOPs) as prompt sequences [9], with agents transmitting structured intermediate artefacts rather than free text, significantly reducing information loss. AutoGen [10] provides a flexible conversational programming paradigm. CAMEL [11] reveals the possibility of emergent cooperative behaviour in LLM multi-agent systems.

However, adapting these paradigms to network resource management faces three specific challenges: first, resource allocation is **competitive constrained optimisation** rather than purely cooperative; second, decisions have **strong real-time constraints**; third, decision outcomes have **clear quantitative evaluation criteria** (SLA satisfaction rate, throughput, latency, etc.).

Multi-agent attempts in the networking domain are still in early stages. CommLLM's three-component architecture provides a conceptual framework but lacks implementation details. The dual-loop edge-terminal collaboration by Qu et al.[6] is the most architecturally informative, but coordination remains primarily centralised. ORAN-GUIDE [8] employs an "LLM provides semantic understanding, RL handles online optimisation" division of labour, offering insights for resolving the LLM decision latency vs. network real-time contradiction.

3.3 Positioning of This Research

As summarised in Table 1, compared with WirelessAgent [1] (single-agent slicing decision) and existing multi-agent concepts without end-to-end slicing validation, this research aims to develop an end-to-end network slice resource allocation framework and proposes an explicit proposal–evaluation–negotiation protocol under two-tier scheduling.

Table 1. Comparison of related frameworks and positioning of this research.

Feature	**WirelessAgent (2025)**	**CommLLM (2024)**	**Qu et al. (2025) [6]**	**This Research**
Agent count	Single agent	Multi-agent (concept)	Dual-loop multi-agent	**Multi-agent**
Validation scenario	Network slicing	Semantic comm.	Edge-terminal collab.	**Network slice resource allocation**
Negotiation mechanism	None	Unspecified	Centralised coord.	**Proposal-evaluation-negotiation protocol**
Real-time handling	No explicit constraint	Not discussed	Inner-loop re-planning	**Two-tier scheduling (LLM low-freq + rule high-freq)**
Simulation validation	Custom env.	Concept level	Partial validation	**Gymnasium standardised env.**

3.4 Reproducibility Status and Open-Source Resources

Through verification of repository accessibility and runnable entry points, the open-source resources used as reproducible references in this work are listed in **Table 2**.

Table 2. Verified open-source resources and reproducibility status.

Resource	Source	Core Function
WirelessAgent_R1	github.com/jwentong/WirelessAgent_R1	LangGraph agent + network slicing KB
network-slicing gym [20]	github.com/jjalcaraz-upct/network-slicing	Gymnasium-based RAN slicing RL env.
CommLLM	github.com/jiangfeibo/CommLLM	Multi-agent LLM framework with retrieval, cooperative planning, and evaluation loop
NetLLM	github.com/duowuyms/NetLLM	Adapts LLMs to networking tasks via low-rank fine-tuning and specialised task heads
LangGraph	github.com/langchain-ai/langgraph	State-machine orchestration library for long-chain, multi-agent LLM workflows

Key findings: Several existing academic frameworks rely on closed-source APIs such as GPT-4 and do not release complete experimental code, while WirelessAgent is among the few works that provide a fully executable and reproducible codebase. Based on recently published repositories and engineering implementations, state-graph-based orchestration frameworks such as LangGraph are frequently adopted in related studies, as they facilitate controllable multi-step reasoning and structured tool invocation. To the best of our knowledge, publicly available datasets specifically designed for network slicing, with explicit SLA annotations and detailed workload descriptions, remain scarce. Consequently, most existing studies rely on synthetic traffic models or generate workload sequences directly within simulation environments.

4. System Architecture Design

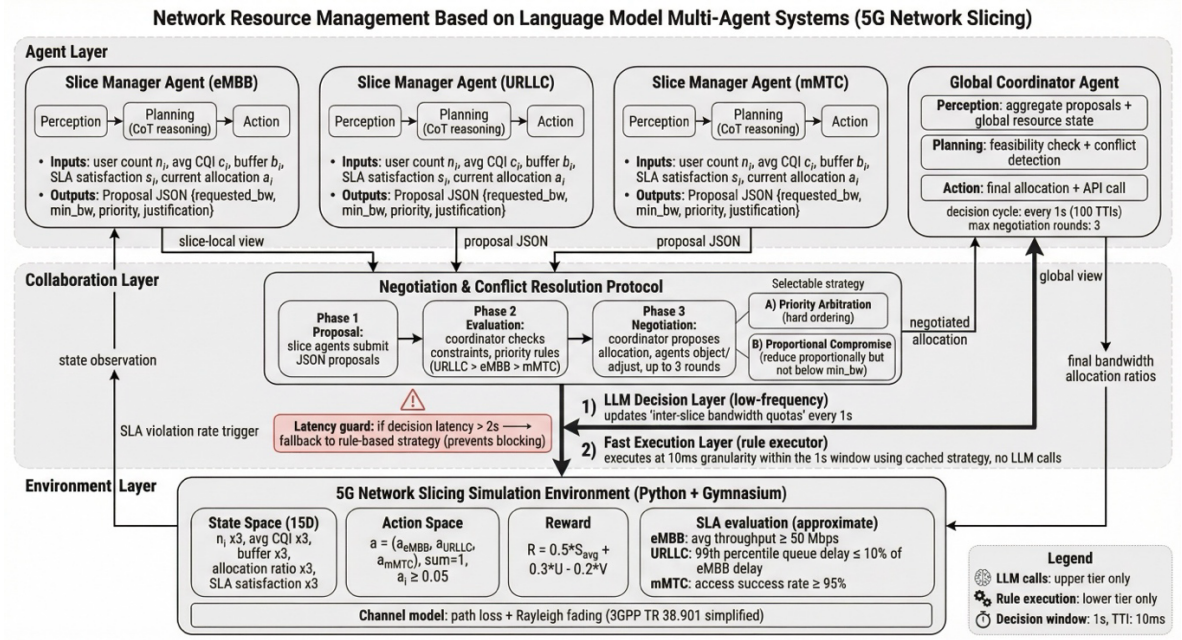


Figure 1. System Architecture

4.1 Overall Architecture and Two-Tier Scheduling
The framework comprises three layers, as shown in **Figure 1**: the **Agent Layer** defines specialised agents and their configurations; the **Collaboration Layer** implements inter-agent negotiation and conflict resolution; the **Environment Layer** provides a standardised simulation environment interface. The entire system is orchestrated via LangGraph, with GPT-4 API as the LLM backbone.

Two-Tier Scheduling Architecture (Core Engineering Design): To address the order-of-magnitude gap between LLM inference latency (hundreds of milliseconds to seconds) and network millisecond-level real-time requirements, this research adopts an explicit two-tier scheduling design:

- **Upper tier (LLM decision layer):** Triggers a multi-agent LLM decision every 1 second (100 TTIs) to update inter-slice bandwidth allocation quotas. Emergency decisions can be triggered early when SLA violation rates exceed a threshold.
- **Lower tier (fast execution layer):** Within each decision window, executes rule-based cached strategies at 10ms granularity (maintaining the upper-tier allocation), with no LLM inference involved.
- **Maximum decision latency threshold:** Set at 2 seconds; if exceeded, the system automatically falls back to rule-based strategies to prevent LLM inference from blocking system operation.

4.2 Agent Design
Each agent contains three core modules — Perception, Planning, and Action:

Slice Manager Agent: One per slice type, three in total. The Perception module converts the slice's numerical state (user count, average CQI, buffer occupancy, current SLA satisfaction rate) into structured natural language descriptions. The Planning module uses Chain-of-Thought (CoT) reasoning [18] to evaluate current resource demands. The Action module outputs resource demand proposals (requested bandwidth, minimum acceptable amount, priority declaration, justification).

Global Coordinator Agent: Only one, responsible for cross-slice coordination. The Perception module aggregates all slice agents' proposals and the global resource state. The Planning module determines whether slice proposals are compatible (whether total demand exceeds total resources); if compatible, proposals are directly approved; if in conflict, the negotiation protocol is triggered. The Action module executes the final bandwidth allocation scheme and calls the simulation environment API.

4.3 Negotiation Protocol Design
When the Global Coordinator Agent detects that slice proposals conflict (total resource demand exceeds available resources), the three-phase negotiation protocol is triggered:
Phase 1 (Proposal): Each Slice Manager Agent independently evaluates demand and submits a resource proposal in structured JSON format.
Phase 2 (Evaluation): The Global Coordinator Agent evaluates global constraint feasibility — checking whether total demand is excessive and whether hard priority rules exist (e.g., URLLC takes precedence over eMBB).
Phase 3 (Negotiation): If conflicts exist, the Global Coordinator Agent proposes an initial allocation scheme; affected Slice Manager Agents may raise objections and alternative proposals, with a **maximum of 3 rounds** to control latency and token costs. Consensus is reached through priority arbitration or proportional compromise.

The experiments will compare two strategies: **Priority Arbitration** (hard ordering: URLLC > eMBB > mMTC) and **Proportional Compromise** (each slice proportionally reduces demand, but not below the minimum acceptable amount).

4.4 Simulation Environment
A custom 5G network slicing simulation environment is built using Python and Gymnasium [20].

State Space (15 dimensions), defined and normalised as in **Table 3:**

Table 3. State space definition and normalisation in the Gymnasium-based environment.

Dimension	Meaning	Range	Normalisation
$n_i(\times 3)$	Current user count per slice	[0, N_max]	/ N_max
$\overline{c_i}(\times 3)$	Average CQI per slice	[1, 15]	/ 15
$b_i(\times 3)$	Buffer occupancy per slice	[0, 1]	Raw value
$a_i(\times 3)$	Current bandwidth allocation ratio per slice	[0, 1]	Raw value
$s_i(\times 3)$	SLA satisfaction rate per slice (sliding window)	[0, 1]	Raw value

Action Space: Continuous vector $a = (a_{eMBB}, a_{URLLC}, a_{mMTC})$, subject to $\sum a_i = 1$ and $a_i \geq 0.05$.

Reward Function: $R = w_S \cdot \bar{S} + w_U \cdot U - w_V \cdot V$, where $\bar{S}$, U, and V are normalised to $[0,1]$, representing the weighted average SLA satisfaction rate, bandwidth utilisation, and SLA violation penalty, respectively. The coefficients w_S, w_U, w_V are tunable hyperparameters satisfying $w_S + w_U + w_V = 1$.

SLA Evaluation (Simulation-Layer Approximate Definitions): The slice-wise SLA conditions used for evaluation are summarised in **Table 4**, and these metrics represent relative priority isolation rather than strict 3GPP end-to-end physical layer metrics.

Table 4. Simulation-layer SLA approximations and design intent per slice type.

Slice Type	SLA Condition	Design Intent	Typical Application
eMBB	Average user throughput ≥ 50 Mbps	High throughput guarantee	Video streaming
URLLC	99th percentile queuing delay ≤ 10% of eMBB's delay	Relative high-priority guarantee	Industrial control
mMTC	Access success rate ≥ 95%	Massive access guarantee	IoT sensors

5. Baseline Method Implementation

The following baseline algorithms have been implemented and initially debugged:

Rule-based strategies: Fixed-ratio allocation (eMBB:URLLC:mMTC = 5:3:2), following commonly adopted traffic composition settings in network slicing simulations [23], and threshold-based dynamic adjustment triggered by SLA violation rates. The former serves as a performance lower bound, while the latter represents common engineering heuristics.

Deep Reinforcement Learning: Proximal Policy Optimisation (PPO) [22] implemented using Stable-Baselines3 with Multi-Layer Perceptron (MLP) policy networks, trained to convergence (early stopping when the average reward change rate over the last 50 episodes < 1%, with a maximum of 1,000 episodes). Training convergence curves are recorded.

Single-Agent LLM: A single GPT-4 agent receives the global network state and outputs resource allocation decisions via CoT reasoning. No multi-agent collaboration. This baseline serves as a direct comparison against the multi-agent framework.

Experimental Design

Experimental Setup

Experimental Setup is fixed across all compared methods, and the full parameter settings are reported in **Table 5.**

Table 5. Experimental configuration for the two-tier LLM multi-agent framework.

Parameter	Setting
System bandwidth	100 MHz (reference NR n78 band)
Slice types	eMBB / URLLC / mMTC
User scale	100 users (standard scenario)
LLM decision cycle	Multi-agent LLM decision triggered every 1 second (100 TTIs)
Lower-tier execution granularity	10ms (TTI-level), rule-based cached strategy within decision windows
Maximum decision latency threshold	2 seconds (automatic fallback to rule-based strategy if exceeded)
Total simulation duration	500 LLM decision cycles per episode

Comparison Methods

Comparison Methods (M1–M5) are summarised in **Table 6.**

Table 6. Summary of compared methods and baselines (M1–M5).

ID	Method	Category	Description
M1	Fixed-Ratio	Rule-based	Fixed ratio allocation (5:3:2)
M2	Threshold-based	Rule-based	Dynamic adjustment based on SLA violation threshold
M3	PPO	Deep RL	MLP policy network, trained to convergence
M4	Single-Agent LLM	Single-agent LLM	GPT-4 + CoT, no collaboration
M5	Multi-Agent LLM	Multi-agent LLM (this work)	Slice agents + coordinator agent + negotiation

Evaluation Metrics

Evaluation Metrics and their computation rules are defined in **Table 7**.

Table 7. Evaluation metrics and computation definitions.

Metric	Computation
Per-slice SLA satisfaction rate	Proportion of decision cycles meeting SLA conditions
Weighted total throughput	Sum of all user throughputs
Bandwidth utilisation	Actual used bandwidth / Total allocated bandwidth
Inter-slice fairness	Jain's fairness index (based on per-slice SLA satisfaction rates)
Per-decision latency	End-to-end time from state input to decision output
API call cost	Total token consumption per episode (input + output)

Experiment 1: Core Performance Comparison (RQ1)

Purpose: Validate the performance differences of the multi-agent LLM framework relative to rule-based strategies, DRL, and single-agent LLM.

Method: Run M1–M5 for both the steady-state scenario (100 users) and the burst scenario (eMBB users doubled at cycle 250). Each setting is executed **30 independent runs with distinct random seeds**; LLM temperature is fixed to 0 to limit variability to environment and DRL exploration.
Core Deliverables: 1 main table (SLA satisfaction rate, throughput, utilisation, fairness, decision latency) + 1 trend curve showing performance under load variation.
Hypotheses and Contingency Analysis: Hypothesis: M5 (multi-agent) outperforms M4 (single-agent) and M3 (PPO). If the hypothesis is not supported, the analysis will examine whether multi-agent communication overhead offsets collaborative benefits, and whether the scenario complexity is insufficient to demonstrate multi-agent advantages.

Experiment 2: Negotiation Mechanism Validation (RQ2)
Purpose: Validate whether the inter-agent negotiation protocol provides structural advantages.
Method: Compare **M5 (full negotiation)** and **M5-no-neg (direct allocation)** under both scenarios, **30 seeds per setting** with the same randomness controls as above.
Core Deliverables: 1 comparison table + 1 inter-slice fairness (Jain index) comparison chart.
Hypotheses and Contingency Analysis: Hypothesis: the negotiation mechanism significantly improves fairness and SLA satisfaction rate under slice conflict scenarios. If not supported, the discussion will address whether centralised dictatorial allocation is already sufficient for simple scenarios, and whether the value of negotiation requires more complex multi-slice conflicts to manifest.

References

[1] J. Tong *et al.*, "WirelessAgent: Large Language Model Agents for Intelligent Wireless Networks," *arXiv.org*, 2025. https://arxiv.org/abs/2505.01074 (accessed Mar. 02, 2026).
[2] J. Shao *et al.*, "WirelessLLM: Empowering Large Language Models Towards Wireless Intelligence," *Journal of Communications and Information Networks*, vol. 9, no. 2, pp. 99–112, Jun. 2024, doi: https://doi.org/10.23919/jcin.2024.10582827.
[3] F. Jiang *et al.*, "Large Language Model Enhanced Multi-Agent Systems for 6G Communications," *IEEE Wireless Communications*, vol. 31, no. 6, pp. 48–55, Aug. 2024, doi: https://doi.org/10.1109/mwc.016.2300600.
[4] D. Wu *et al.*, "NetLLM: Adapting Large Language Models for Networking," Aug. 2024, doi: https://doi.org/10.1145/3651890.3672268.
[5] Y. Chen *et al.*, "NetGPT: An AI-Native Network Architecture for Provisioning Beyond Personalized Generative Services," *IEEE Network*, pp. 1–1, Jan. 2024, doi: https://doi.org/10.1109/mnet.2024.3376419.
[6] Z. Qu, W. Wang, Z. Yu, B. Sun, Y. Li, and X. Zhang, "LLM Enabled Multi-Agent System for 6G Networks: Framework and Method of Dual-Loop Edge-Terminal Collaboration," *arXiv.org*, 2025. https://arxiv.org/abs/2509.04993 (accessed Mar. 02, 2026).
[7] H. Chergui, M. C. Cid, K. P. Sayyad, D. C. Mur, and C. Verikoukis, "Toward an Unbiased Collective Memory for Efficient LLM-Based Agentic 6G Cross-Domain Management," *arXiv.org*, 2025. https://arxiv.org/abs/2509.26200 (accessed Mar. 02, 2026).
[8] F. Lotfi, H. Rajoli, and F. Afghah, "ORAN-GUIDE: RAG-Driven Prompt Learning for LLM-Augmented Reinforcement Learning in O-RAN Network Slicing," *arXiv.org*, 2025. https://arxiv.org/abs/2506.00576 (accessed Mar. 02, 2026).
[9] S. Hong *et al.*, "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," *arXiv.org*, Aug. 07, 2023. https://arxiv.org/abs/2308.00352
[10] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," *arXiv.org*, Oct. 03, 2023. https://arxiv.org/abs/2308.08155
[11] G. Li, A. Kader, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative Agents for 'Mind' Exploration of Large Language Model Society," *arXiv.org*, 2023. https://arxiv.org/abs/2303.17760
[12] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," *arXiv.org*, Jul. 18, 2023. https://arxiv.org/abs/2307.07924
[13] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network Slicing and Softwarization: A Survey on Principles, Enabling Technologies, and Solutions," *IEEE*

Communications Surveys & Tutorials, vol. 20, no. 3, pp. 2429–2453, 2018, doi: https://doi.org/10.1109/comst.2018.2815638.
[14] R. Li *et al.*, "Deep Reinforcement Learning for Resource Management in Network Slicing," *IEEE Access*, vol. 6, pp. 74429–74441, 2018, doi: https://doi.org/10.1109/access.2018.2881964.
[15] V. Sciancalepore, X. Costa-Perez, and A. Banchs, "RL-NSB: Reinforcement Learning-Based 5G Network Slice Broker," *IEEE/ACM Transactions on Networking*, vol. 27, no. 4, pp. 1543–1557, Aug. 2019, doi: https://doi.org/10.1109/tnet.2019.2924471.
[16] L. Wang *et al.*, "A Survey on Large Language Model based Autonomous Agents," *arXiv.org*, Sep. 07, 2023. https://arxiv.org/abs/2308.11432v2
[17] H. Zhou *et al.*, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities," *IEEE Communications Surveys & Tutorials*, pp. 1–1, Jan. 2024, doi: https://doi.org/10.1109/comst.2024.3465447.
[18] J. Wei *et al.*, "Chain of Thought Prompting Elicits Reasoning in Large Language Models," *arXiv:2201.11903 [cs]*, Oct. 2022, Available: https://arxiv.org/abs/2201.11903
[19] P. Caballero, A. Banchs, G. De Veciana, and X. Costa-Perez, "Network Slicing Games: Enabling Customization in Multi-Tenant Mobile Networks," *IEEE/ACM Transactions on Networking*, vol. 27, no. 2, pp. 662–675, Apr. 2019, doi: https://doi.org/10.1109/tnet.2019.2895378.
[20] G. Brockman *et al.*, "OpenAI Gym," *arXiv.org*, 2016. https://arxiv.org/abs/1606.01540
[21] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Neural Information Processing Systems*, 2020. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimisation Algorithms," *arXiv.org*, Aug. 28, 2017. https://arxiv.org/abs/1707.06347
[23] A. R. Nasser and O. Y. Alani, "Investigation of Multiple Hybrid Deep Learning Models for Accurate and Optimized Network Slicing," Computers, vol. 14, no. 5, p. 174, May 2025, doi: https://doi.org/10.3390/computers14050174.

尚需完成的任务 Work to do:

1. **Complete Gymnasium simulation environment:** Implement the 15-dimensional state space, action space, reward function, and SLA evaluation logic. Ensure rule-based baseline (M1) runs correctly and outputs all metrics.
2. **Minimal dual-agent version:** Implement 1 slice agent + 1 coordinator agent as a minimal end-to-end pipeline via LangGraph, completing the full "perceive → propose → evaluate → allocate → execute" flow.
3. **First round of baseline comparison data:** Run at least M1 (rules), M3 (PPO), and M4 (single-agent LLM) comparisons to produce 1 core metrics table.
4. **Full multi-agent framework:** Extend to 3 slice agents + 1 coordinator agent with the negotiation protocol implemented.
5. **Complete execution of Experiments 1 and 2:** Finish all core comparisons and negotiation mechanism validation.
6. **Report writing:** Complete all chapters, with emphasis on figure/table quality for experimental results and honest Discussion.

存在问题 Problems:

1. **LLM decision latency vs. network real-time requirements**: LLM inference latency (hundreds of milliseconds to seconds) is orders of magnitude greater than network millisecond-level real-time requirements. Multi-agent scenarios further amplify latency through multiple inter-agent calls. The two-tier scheduling architecture (§4.1) has been adopted in response, but its practical effectiveness remains to be validated.

2. **API call costs**: GPT-4 requires multiple API calls per decision in multi-agent negotiation scenarios, with significant token consumption and costs per episode. Total costs must be estimated early, and a budget cap established.
3. **Evaluation data authenticity**: The simulation environment is based on synthetic traffic models with queuing theory approximations for SLA evaluation, which may not fully represent real network scenarios.
4. **Schedule risk**: The core multi-agent framework is not yet fully implemented; the remaining period requires simultaneous system implementation, experiments, and report writing. This has been mitigated through scope convergence (2 RQs, 2 core experiments).

拟采取的办法 Solutions:

1. **Two-tier architecture validation**: The experiments will compare "pure rule-based strategy performance within the 1-second window" against "performance improvement after LLM updates" to quantify the marginal value of LLM decisions, with a 2-second maximum decision latency threshold as a deployment feasibility criterion.
2. **Cost control**: Estimate per-episode token consumption limits from trial runs before experiments; set a total budget. The negotiation round hard limit of 3 rounds controls token costs at the mechanism level.
3. **Honest discussion of limitations**: The Discussion chapter will explicitly list Threats to Validity — simulation approximation boundaries, relative SLA definition limitations, and synthetic data representativeness.

论文结构 Structure of the final report: (Chapter headings and section sub headings)

北京邮电大学 本科毕业设计（论文）教师指导记录表

Project Supervision Log

学院 School	International School	**专业 Programme**	**Intelligent Science and Technology**		
姓 Family name	Zhao 赵	**名 First Name**	Zheyun 哲昀		
BUPT 学号 BUPT number	2022213670	**QM 学号 QM number**	221170559	**班级 Class**	2022215124
论文题目 Project Title	Network Resource Management Based on Language Model Multi-Agent Systems				

Please record supervision log using the format below:

Date: dd-mm-yyyy
Supervision type: face-to-face meeting/online meeting/email/messages/other (please specify)
Summary:

Date: 11-12-2025
Supervision type: online meeting
Summary: Initial online meeting to discuss research direction and expectations for the project.

Date: 14-12-2025
Supervision type: messages
Summary: Sent several research papers to the supervisor and clarified the specific task requirements.

Date: 09-01-2026
Supervision type: messages
Summary: Reported recently read papers and shared brief insights from the literature review.

Date: 12-01-2026
Supervision type: face-to-face meeting
Summary: In-person meeting to discuss relevant papers and their implications for the research topic.

Date: 15-01-2026
Supervision type: face-to-face meeting
Summary: Continued discussion of related papers and potential research directions.

Date: 20-01-2026
Supervision type: online meeting
Summary: Discussed experimental progress and identified several possible directions for optimisation.

Date: 22-01-2026
Supervision type: messages
Summary: Follow-up discussion regarding experiments and implementation details.

Date: 29-01-2026
Supervision type: messages

Summary: Received general guidance from the supervisor, emphasising critical thinking, extensive reading, and conducting more experiments.

Date: 30-01-2026
Supervision type: messages
Summary: Reported additional papers that had been read during the literature review.

Date: 02-02-2026
Supervision type: messages
Summary: Continued reporting progress on literature review and summarising key findings.

Date: 08-02-2026
Supervision type: messages
Summary: Shared updates on newly read papers and ongoing literature study.

Date: 10-02-2026
Supervision type: messages
Summary: Reported recently read papers and submitted reading notes for feedback.

Date: 02-03-2026
Supervision type: messages
Summary: Sent the draft of the mid-term report to the supervisor for review.

Date: 05-03-2026
Supervision type: messages
Summary: Sent the slides prepared for the mid-term presentation.

Additional appendices

Prompt Templates for Each Agent Role

The following templates are used by the slice manager agents, the global coordinator agent, and the single-agent LLM baseline. The running code uses Chinese wording; this appendix gives faithful English translations for the report. Placeholders in braces are filled at runtime by LangChain (`{state_description}`, `{slice_name}`, etc.).

Slice manager agents (per-slice system prefix)

Each slice agent receives one of the role-specific strings below as `{system_prompt}`, combined with the shared proposal or counter-proposal instructions.

```
eMBB:
You are the eMBB slice manager agent, responsible for high-bandwidth mobile
broadband (video streaming, large downloads).
SLA: average user throughput >= 50 Mbps.

URLLC:
You are the URLLC slice manager agent, responsible for ultra-reliable
low-latency communication (industrial control, remote surgery).
SLA: 99th percentile queuing delay <= 10% of eMBB delay. URLLC has the
highest priority.

mMTC:
You are the mMTC slice manager agent, responsible for massive machine-type
communication (IoT sensor networks).
SLA: access success rate >= 95%.
```

Slice manager — initial proposal (system + human)

```
[System]
{system_prompt}

Use chain-of-thought reasoning on the current slice state, then output a
JSON resource request proposal.
Output JSON only (no other text):
{"requested": 0.xx, "minimum": 0.xx, "priority": "high/medium/low",
 "justification": "brief reason"}

[Human]
Current network state:
{state_description}
```

```
Propose a resource request for the {slice_name} slice.
```

Slice manager — counter-proposal after coordinator feedback

```
[System]
{system_prompt}

Adjust your resource request using the coordinator feedback. Output JSON
only (no other text):
{"requested": 0.xx, "minimum": 0.xx, "priority": "high/medium/low",
 "justification": "brief reason"}

[Human]
Current network state:
{state_description}

Coordinator feedback:
{coordinator_feedback}

Revise your resource request for the {slice_name} slice.
```

Global coordinator agent

```
[System]
You are the global coordinator for 5G network slices, responsible for
cross-slice resource coordination.

Responsibilities:
1. Collect resource proposals from each slice agent
2. Check global constraints (bandwidth fractions sum to 1.0;
   minimum 0.05 per slice)
3. Decide whether proposals are compatible (whether total demand
   exceeds capacity)
4. If there is conflict, propose an allocation

Priority order: URLLC > eMBB > mMTC

Output JSON only (no other text):
{"compatible": true/false,
 "allocation": {"eMBB": 0.xx, "URLLC": 0.xx, "mMTC": 0.xx},
 "feedback": "rationale or per-slice feedback"}

[Human]
```

```
Current network state:
{state_description}

Per-slice proposals:
{proposals_text}

Total requested bandwidth fraction: {total_requested:.2f} (available: 1.00)

Assess compatibility and provide an allocation.
```

The field `proposals_text` is assembled in code as one line per slice, e.g. `[eMBB] requested=0.xx, minimum=0.xx, priority=..., justification=...`.

Single-agent LLM baseline (method M4)

```
[System]
You are a 5G network slicing resource expert. Given the current network
state, assign bandwidth fractions to the three slices.

SLA requirements:
- eMBB: average user throughput >= 50 Mbps
- URLLC: 99th percentile queuing delay <= 10% of eMBB delay
- mMTC: access success rate >= 95%

Constraints: three fractions sum to 1.0; minimum 0.05 per slice;
priority URLLC > eMBB > mMTC

After chain-of-thought analysis, output JSON only (no other text):
{"eMBB": 0.xx, "URLLC": 0.xx, "mMTC": 0.xx}

[Human]
Current network state:
{state_description}

Provide a bandwidth allocation.
```

Agent Message Format (JSON Schema)

This appendix summarises the structured JSON messages exchanged between the LLM agents and the LangGraph orchestration layer. The environment's natural-language state is emitted in Chinese; field meanings below are given in English for clarity.

Environment state description (natural language input)

The environment builds a single string passed to every LLM call as `{state_description}`. Each line has the following semantics (runtime text is Chinese; structure is fixed):

- Header: current decision step index and total system bandwidth (MHz).
- One block per slice `eMBB`, `URLLC`, `mMTC`: user count, average CQI, buffer occupancy in $[0, 1]$, current bandwidth fraction and absolute MHz, sliding-window SLA satisfaction rate in $[0, 1]$.

Slice manager agent — proposal object

Parsed by `JsonOutputParser`; keys are normalised before downstream use. Intended shape:

```
{
  "slice": "eMBB" | "URLLC" | "mMTC", // injected after parsing
  "requested": <float>, // requested bandwidth fraction
  "minimum": <float>, // >= 0.05 after validation
  "priority": "high" | "medium" | "low",
  "justification": "<string>"
}
```

The `slice` field is appended in code so downstream components always know the source slice. Defaults on parse failure are defined per slice.

Global coordinator agent — evaluation object

Returned by the coordinator LLM; allocation is renormalised to sum to 1.

```
{
  "compatible": <bool>,
  "allocation": {
    "eMBB": <float>,
    "URLLC": <float>,
    "mMTC": <float>
  },
  "feedback": "<string>"
```

```
}
```

When building the coordinator prompt, each proposal is flattened to one line of text for `{proposals_text}`, e.g. `[eMBB] requested=0.xx, minimum=0.xx, priority=..., justification=...`.

Single-agent baseline (M4) — allocation object

Direct mapping from JSON to the three-dimensional action (then clipped to ≥ 0.05 and normalised):

```
{ "eMBB": <float>, "URLLC": <float>, "mMTC": <float> }
```

LangGraph multi-agent state (`MASState`)

Typed dictionary carried across nodes in the LangGraph workflow:

Field	Role
`state_description`	Natural-language snapshot from the environment (see above).
`proposals`	List of slice proposal dicts (latest round).
`coordinator_result`	Coordinator evaluation dict; `compatible` drives routing.
`negotiation_round`	Integer; incremented after each negotiation node visit.
`max_rounds`	Cap on negotiation iterations (default 3).
`strategy`	`"priority"` or `"proportional"`: selects Priority Arbitration vs. Proportional Compromise.
`final_allocation`	Final dict `{"eMBB": float, "URLLC": float, "mMTC": float}` passed to the env.
`total_tokens`	Heuristic token-cost accumulator for logging.
`is_resolved`	Set when negotiation applies a resolution.

Routing and final action

If `coordinator_result.compatible` is true, the graph goes to the finalise node and uses the coordinator's `allocation`. Otherwise the negotiation node computes an allocation from the current `proposals` using the selected strategy; the graph may loop back to slice agents until `negotiation_round >= max_rounds` or resolution. The finalise node outputs a dict with keys `eMBB`, `URLLC`, `mMTC` (floats, interpreted as fractions, then clipped and renormalised to sum to 1).

The no-negotiation variant (M5-no-neg) omits the negotiation loop; the final action is taken from the coordinator's `allocation` only.

DRL Training Convergence Curves

Figure 10 shows the episode reward trajectory during Proximal Policy Optimisation (PPO) training for method M3. The agent was trained for up to 2 000 episodes with an early-stopping criterion (improvement < 0.005 over a 100-episode window). Training converged after approximately 250 episodes, reaching a plateau around an episode reward of 38–40.

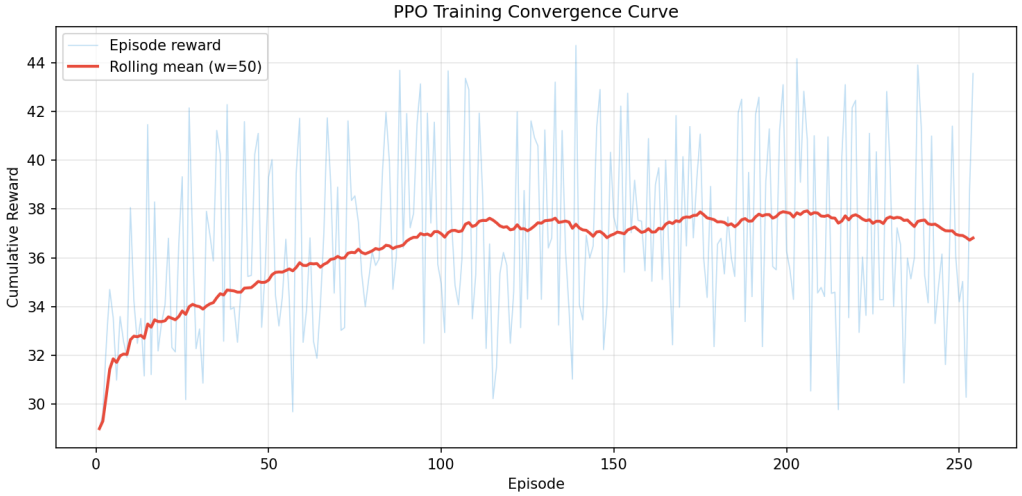


Figure 10: PPO training curve: episode reward versus training episode. The shaded region indicates ± 1 standard deviation computed over a rolling window. Training was performed with seed 42, learning rate 3×10^{-4}, 1 024 rollout steps, batch size 64, and 10 PPO epochs per update.

Risk and Environmental Impact Assessment

Likelihood L, consequence C, and risk $R = L \times C$ are scored as in the QMUL handbook (Sect. 3.6.12). The subsections below give the project-specific analysis.

1. Risks to successful project completion

The work combines a custom Gymnasium slicing simulator, Stable-Baselines3 PPO training, and LangGraph-orchestrated multi-agent LLM policies with external API calls. The main risks are summarised as Items R1–R5.

R1 — LLM API outage, quota, or rate limiting L=3, C=3, R=9 (significant). Mitigation: automatic retry logic with exponential back-off and configurable timeouts; cached intermediate results after each seed to enable resumption; fallback allocation maintained in all agent classes; rule-based and PPO baselines documented partial results if a full sweep cannot finish.

R2 — Underestimated experiment and write-up time L=3, C=3, R=9 (significant). Mitigation: prioritised core method comparisons (M1–M5) before optional ablations; reduced exploratory seeds after pilot runs; froze scope after an agreed milestone.

R3 — Implementation or configuration error affecting validity L=2, C=4, R=8 (significant). Mitigation: fixed random seeds and centralised configuration; version control; spot-checked metrics against baselines; retained aggregation scripts for reproducibility.

R4 — Dependency or environment drift L=2, C=2, R=4 (moderate). Mitigation: pinned dependencies; recorded Python and package versions; reran smoke tests before final data collection.

R5 — Loss of source code or experimental data L=1, C=5, R=5 (moderate). Mitigation: all code and data were version-controlled on GitHub with regular pushes; experimental CSV outputs were committed after each experiment batch.

Malfunctioning software (crashes mid-experiment) is treated as a moderate operational risk: logging, smaller batches, and resumable scripts mitigate potential disruption.

2. Risks of harm to people and/or animals

The project is *computational*: simulation and API-based inference only; there are no human participants, animals, or field trials. If the proposed framework were deployed in a real URLLC slice, misallocation could disrupt safety-critical services, but this risk is entirely outside the

scope of the current simulation study (L=1, C=4, R=4, moderate). Residual ergonomics risk from prolonged screen work is rated L=3, C=1, R=3 (low), mitigated by regular breaks and an ergonomic workstation. No further institutional ethics procedures apply beyond normal safe computing practice.

3. Environmental impact

Energy and emissions. Training PPO and running experiments uses local CPU/GPU electricity; LLM calls consume remote data-centre energy. Avoidable extra compute and API calls are rated at L=3, C=2, R=6 (moderate): pilot runs preceded large sweeps, redundant invocations were avoided, cached decisions were reused where possible, and `temperature=0` was used to limit stochastic variance.

Carbon footprint of DRL training. PPO training completed in approximately 30 minutes on a single GPU; the environmental impact is negligible compared to large-scale deep learning workloads (L=2, C=1, R=2, low).

Waste and hardware. No chemical or radioactive waste; no dedicated hardware beyond a standard workstation; end-of-life equipment follows institutional recycling (L=1, C=1, R=1, no risk).

4. Financial loss

The main exposure is pay-per-use cloud LLM charges at scale. Uncontrolled seeds, episodes, or debugging runs can inflate cost (L=3, C=2, R=6, moderate). Mitigation: budget cap, monitored token usage, temperature zero to limit retries, and cheaper models for dry-run debugging. All DRL training was performed locally; LLM API costs were tracked and remained within the free-tier and educational credit allowances. No contractual liability to third parties arises from the simulation-only scope (L=1, C=2, R=2, low).

Summary

The highest-priority contingencies are (i) experiments remaining feasible when the LLM API is constrained (R=9), (ii) calendar and API budget control so core comparisons finish on time (R=9), and (iii) validity through configuration control and sanity checks (R=8). All three are classified as *Significant Risk* and have been addressed through retry/caching mechanisms, scope management, and systematic testing, respectively. No risk was assessed at the *High Risk* level ($R \geq 15$). Human, animal, and acute environmental harm are negligible for this software-only study; energy and API cost are managed as low-to-moderate concerns.